운영 에이전트 & 감사 에이전트

파이프라인 자율 진단 체계 설계

Design Document 11

2026-04-10 | 초안 (v0.1)

1 개요

09장의 감사/거버넌스 인프라(audit_logger, fairness_monitor, compliance_checker 등)와 08장의 추천사유 생성(async_orchestrator, self_checker)은 개별 컴포넌트로 존재한다. 이 문서는 그 컴포넌트들을 두 개의 자율 진단 에이전트로 묶어, 파이프라인 전체를 관점별로 관찰하고 담당자에게 "어디를 봐야 하는지"를 진단하는 체계를 설계한다.

> 핵심 설계 원칙
> "뭘 새로 만들까"가 아니라 "기존 컴포넌트를 어떤 관점으로 묶어서 진단하게 할 것인가"가 핵심이다. 기존 인프라를 최대한 재사용하고, 수집 → 진단 → 리포트의 오케스트레이션 레이어만 신규 개발한다.

2 전체 아키텍처

2.1 5-에이전트 체계에서의 위치

Paper 2의 5-에이전트 아키텍처(서빙 3 + 운영 2)에서 운영/감사 에이전트는 서빙 경로와 비동기로 분리된다.

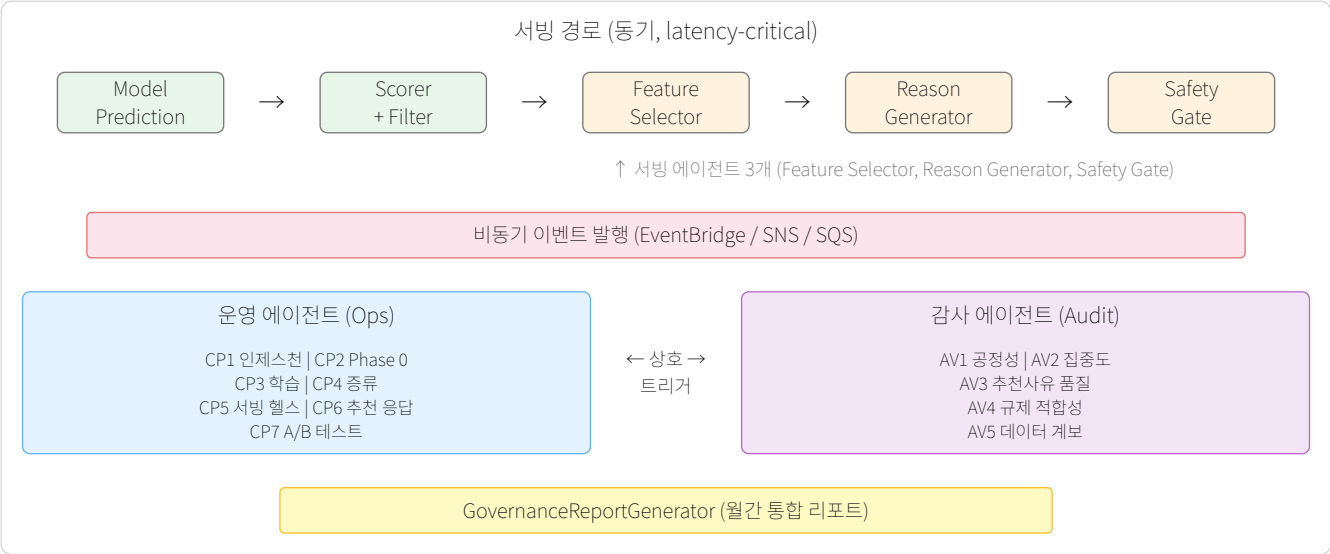


Figure 1: 5-에이전트 아키텍처 전체 구조. 상단 서빙 경로(동기)와 하단 진단 경로(비동기)의 분리.

2.2 비동기 분리의 근거

근거	설명
레이턴시 디커플링	감사 로깅(HMAC chain, DynamoDB)이나 fairness 계산 장애가 추천 응답을 블로킹하지 않음
규제 독립성	감사 정책 변경 시 추천 파이프라인 재배포 불필요 — 릴리즈 사이클 독립
장애 격리	감사 로그 유실 방지를 위한 별도 재시도/DLQ 전략 적용 가능

3 공통 실행 모델: Collect → Diagnose → Report

두 에이전트 모두 동일한 3단계 루프를 따른다. 차이는 어떤 체크포인트를 보는가와 어떤 기준으로 진단하는가이다.

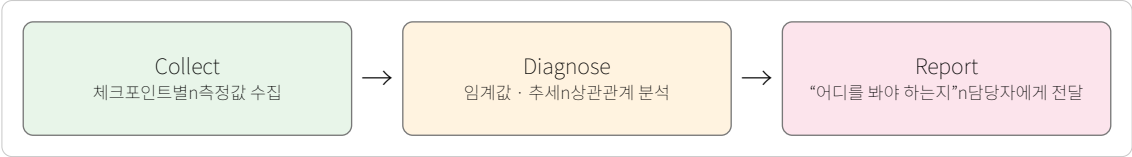


Figure 2: 공통 3단계 실행 루프.

3.1 실행 계층: 온프렘 기본 + AWS Bedrock 확장

3.1.1 설계 철학

에이전트의 본질적 가치(자동 진단 + 체크리스트 판정 + 리포팅)는 결정론적 Python 룰 엔진만으로 완결된다. LLM 없이 온프렘에서 100% 작동하는 것이 기본이다.

AWS 환경에서는 Bedrock을 통해 담당자 편의 기능을 추가한다: 진단 결과를 두고 대화하거나, 변경 영향도를 추론하는 것. 이는 요즘 퍼블릭 클라우드로 추천 서비스를 구성하는 기업들에게 운영 편의성 측면에서 실질적인 가치를 제공하는 레퍼런스 모델이다.

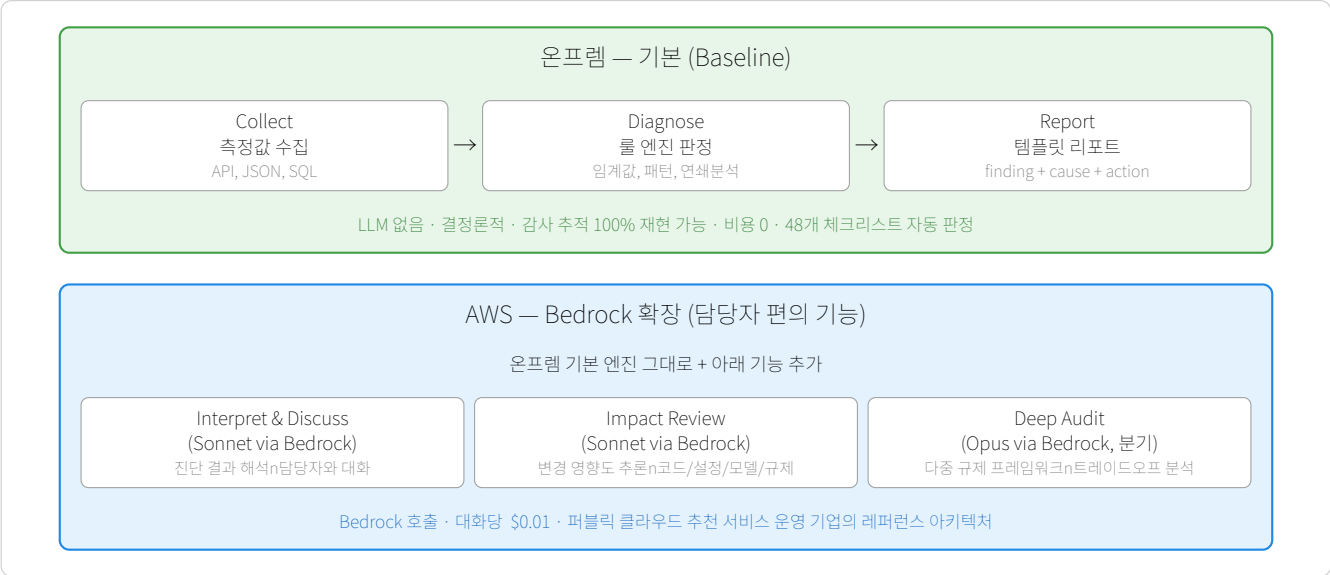


Figure 3: 온프렘 기본 + AWS Bedrock 확장. 온프렘이 완결된 baseline이고, AWS는 편의 기능을 추가.

3.1.2 환경별 기능 매트릭스

기능	온프렘	AWS	비고
체크리스트 자동 판정 (48항목)	O	O	Python 룰 엔진
연쇄 영향 분석 (cross-checkpoint)	O	O	사전 정의 룰 테이블
정형 리포트 (finding/cause/action)	O	O	템플릿 + 수치 삽입
상호 트리거 (Ops ↔ Audit)	O	O	이벤트 기반
거버넌스 리포트 통합	O	O	GovernanceReportGenerator
인시던트 에스컬레이션	O	O	SNS / 이메일 / Slack
진단 결과 해석 대화	–	O	Sonnet via Bedrock

기능	온프렘	AWS	비고
변경 영향도 리뷰	–	O	Sonnet via Bedrock
분기 심층 감사 리뷰	–	O	Opus via Bedrock (선택)

3.1.3 왜 이 구조인가

온프렘이 기본인 이유
(1) 결정론성: 룰 엔진의 진단은 100% 재현 가능 — 같은 입력이면 같은 판정. 감사 추적에서 이것이 LLM 대화보다 본질적으로 중요하다.
(2) 독립성: 외부 API 의존 없이 운영 가능 — 네트워크 단절, 보안 정책, 비용 제약에 무관.
(3) 충분성: 48개 체크리스트 + 연쇄 영향 룰 + 정형 리포트면 대부분의 운영/감사 상황을 커버한다.

AWS Bedrock 확장의 가치
(1) 대화: "이 DI 0.68이 필터 문제인지 모수 문제인지"를 에이전트와 논의할 수 있다 — 담당자가 수치만 보고 판단하는 것보다 의사결정이 빠르다.
(2) 영향도 리뷰: 코드 변경의 하류 영향을 에이전트가 추론해주면 리뷰 부담이 줄고 누락 위험이 감소한다.
(3) 레퍼런스: 퍼블릭 클라우드에서 추천 서비스를 운영하는 기업들에게 "Bedrock 연동으로 운영 에이전트를 이렇게 강화할 수 있다"는 실용적 모델을 제시.

3.2 3-에이전트 합의 메커니즘 (AWS Bedrock)

LLM 기반 해석에는 할루시네이션 리스크가 있다. 같은 진단 결과를 주더라도 Sonnet이 다른 해석을 내릴 수 있다. 이를 구조적으로 완화하기 위해 3개의 독립 에이전트 세션을 병렬 실행하고 합의 수준에 따라 결과를 분류한다.

Figure 4: 3-에이전트 합의 메커니즘. 만장일치, 다수 이상, 소수 의견(마이너리티 리포트)으로 분류.

3.2.1 합의 판정 규칙

α	β	γ	판정
PASS	PASS	PASS	Consensus Pass — 안전. 룰 엔진 판정과 일치 확인만.
WARN	WARN	WARN	Consensus WARN — 확정. 최우선 리뷰. 3개 에이전트의 cause 비교하여 종합 원인 도출.
WARN	WARN	PASS	최우선 리뷰 — 2/3 이상 징후. α, β의 cause를 병합하여 리뷰 항목 생성.
WARN	PASS	PASS	마이너리티 리포트 — 1/3 소수 의견. α의 근거를 별도 기록, 2순위 리뷰 리스트에 추가.

α	β	γ	판정
FAIL	*	*	1개라도 FAIL이면 즉시 에스컬레이션 — FAIL은 합의와 무관하게 최우선.

3.2.2 마이너리티 리포트 (Minority Report)
소수 의견(1/3)이라고 무시하지 않는다. "3명의 프리콕 중 1명만 다른 예측을 했다"는 그 자체로 중요한 신호이다.

마이너리티 리포트에 포함되는 정보:
- 소수 의견 에이전트의 finding + likely_cause + 근거
- 다수 의견과의 구체적 차이점 (어떤 수치를 다르게 해석했는가)
- 과거 마이너리티 리포트 중 사후에 맞았던 비율 (케이스 스토어 통계)

> 마이너리티 리포트의 가치
> 소수 의견이 나중에 맞는 경우가 있다 — 특히 새로운 유형의 문제는 기존 패턴에 익숙한 다수가 놓치고 다른 관점의 소수가 먼저 포착할 수 있다.
> 마이너리티 리포트를 체계적으로 기록하고 사후 검증하면, 합의 메커니즘 자체의 품질을 개선하는 피드백 루프가 된다.
> 케이스 스토어에 `consensus_type: "minority"` 태그로 저장하여 "마이너리티가 맞았던 비율"을 추적한다.

3.2.3 환경별 합의 방식: 독립 투표 vs 순차 심의
AWS와 온프렘에서 합의 메커니즘의 방식 자체가 다르다. AWS는 강한 모델 소수의 독립 투표, 온프렘은 약한 모델 다수의 순차 심의.

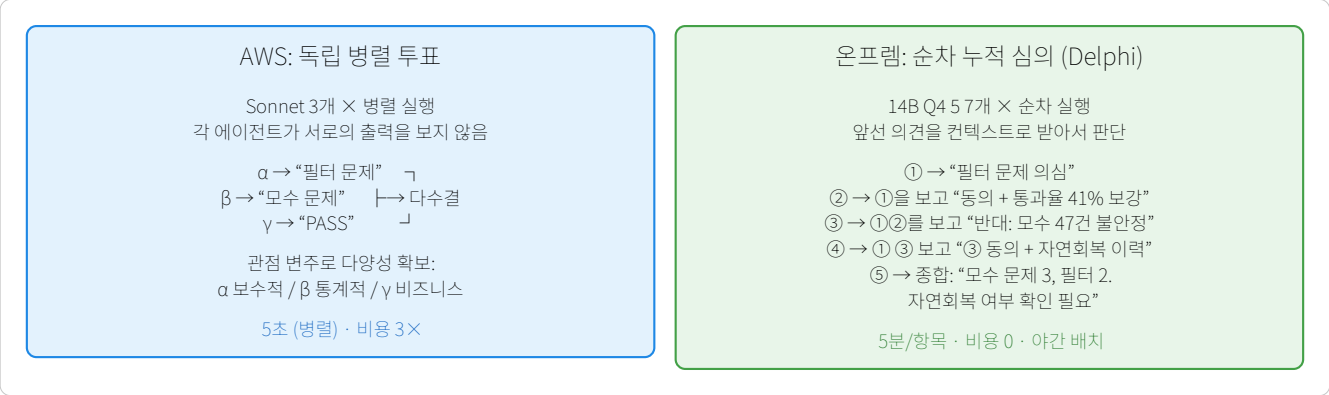


Figure 5: 환경별 합의 방식. AWS는 독립 투표(배심원), 온프렘은 순차 심의(델파이).

3.2.3.1 온프렘: 2-Round 하이브리드 (독립 투표 → 순차 심의)
순수 델파이(순차 심의)는 수렴 편향이 있다 — 뒤로 갈수록 앞 의견에 끌려가서 소수 의견이 사라진다. 특히 14B 같은 약한 모델은 앞에 강한 논거가 있으면 반론을 포기하기 쉽다.

운영/감사 영역에서는 "놓치는 것"이 "오탐"보다 훨씬 위험하다. 금감원이 "왜 이걸 못 잡았나?"라고 물으면 답이 없기 때문이다.

따라서 온프렘에서는 2-Round 하이브리드를 사용한다:

Figure 6: 온프렘 2-Round 하이브리드. Round 1이 마이너리티를 보존하고, Round 2가 논거를 보강.

3.2.3.2 마이너리티 보존 규칙

> 핵심 원칙: Round 1에서 확정된 마이너리티는 삭제되지 않는다
> Round 2에서 ⑥이 "계절 패턴은 타당성 낮다"고 평가하더라도, 마이너리티 리포트에서 삭제되지 않는다.
> 대신 "⑥이 타당성 낮다고 평가 — 근거: … / 원 의견(⑤) 보존"으로 기록된다.
> 최종 판단은 사람이 한다. 에이전트는 소수 의견을 기록하고 근거를 명확히 하는 것까지만 책임진다.

3.2.3.3 2-Round가 순수 델파이보다 나은 이유

측면	순수 순차 심의 (델파이)	2-Round 하이브리드
마이너리티 보존	❌ 뒤로 갈수록 수렴, 소수 의견 소멸	✓ Round 1에서 독립 확보, 이후 삭제 불가
논거 품질	✓ 누적으로 풍부	✓ Round 2에서 보강 (동일 효과)
약한 모델 적합성	△ 앞 의견에 끌림 (conformity bias)	✓ Round 1은 독립, Round 2만 참조
감사 적합성	❌ "왜 소수 의견이 사라졌나" 설명 곤란	✓ 모든 의견 보존, 감사 추적 가능

3.2.3.4 에이전트 수 및 소요시간

3.2.3.5 에이전트 출력 사양

각 에이전트의 출력은 구조화 판정(짧고 정형, 파싱용)과 자유 논거(풍부하고 상세, 사람 + 케이스 스토어용)로 분리한다. 논거는 풍부할수록 좋다 — 담당자의 판단 재료가 되고, 케이스 스토어에 쌓였을 때 유사 검색과 통계 분석의 품질이 올라간다.

```
{
  "verdict": "WARN",
  "confidence": 0.75,

  "reasoning": "이 그룹의 DI 0.68은 임계값 0.80을 명확히 하회한다.
    다만 모수가 47건으로 전체 12,340건의 0.38%에 불과하여,
    필터 1개 추가/제거만으로도 DI가 ±0.15 변동할 수 있다.

    전체 평균 filter 통과율 67% 대비 해당 그룹은 41%로,
    eligibility 필터에서 주로 탈락하고 있다.
```

```
    이는 income 기반 적격성 조건이 해당 그룹에
    구조적으로 불리하게 작용함을 시사한다.

    다만 지난 1월에도 유사한 수치(DI 0.71)가 관찰되었고
    별도 조치 없이 2월에 0.84로 회복한 이력이 있어,
    계절적 요인의 가능성도 배제할 수 없다.",

  "recommendation": "filter 통과율의 그룹별 분해 분석을 먼저 수행하고,
    계절 패턴 여부는 3개월 추이로 판단할 것."
}
```

필드	토큰	용도
`verdict` + `confidence`	10	자동 집계 · 합의 판정용
`reasoning`	300 600	담당자 판단 재료 · 케이스 스토어 임베딩 · 마이너리티 근거
`recommendation`	50 100	suggested_action · 대응 추적용

3.2.3.6 소요시간

입력 1,000 토큰, 출력 500 800 토큰 기준. 14B Q4 on RTX 4070에서 건당 30 40초.

환경	모델	R1	R2	합계	항목당 소요
AWS	Sonnet	3 (병렬)	–	3	5초
온프렘 기본	14B Q4	5 × 35초	2 × 45초	7	4.5분
온프렘 고위험	14B Q4	7 × 35초	2 × 45초	9	5.5분

실제 운용에서는 48개 전체에 합의를 돌릴 필요 없이, 룰 엔진이 WARN/FAIL로 판정한 항목만 (보통 5 10개) 합의를 실행:

시나리오	온프렘	AWS
WARN/FAIL만 (10항목)	45분	1분
고위험 항목만 (5항목)	25분	30초

점검 직후 바로 실행해도 충분한 수준이다. 풍부한 논거가 시간 대비 가치가 높다 — 케이스 스토어에 "왜 이렇게 판단했는지"가 상세히 남으면 6개월 후 유사 케이스 검색 시 질이 완전히 다르다.

3.2.3.7 Round 1 프롬프트 (독립 투표)

```
## 진단 대상
체크항목 4.7: 교차 보호속성 DI 임계값
현재 값: elderly ∩ low_income DI = 0.68 (임계값 0.80)
룰 엔진 판정: WARN

## 참고 데이터
- 전체 평균 DI: 0.85
- 해당 그룹 추천 건수: 47건 / 전체 12,340건
- filter 통과율: 해당 그룹 41% / 전체 67%
- 과거 유사 케이스: [케이스 스토어 검색 결과 첨부]

## 지시
이 WARN 판정에 대해 독립적으로 판단하세요.
PASS/WARN/FAIL 중 하나를 선택하고 근거를 제시하세요.
```

3.2.3.8 Round 2 프롬프트 (순차 심의)

```
## 진단 대상
[Round 1과 동일]

## Round 1 투표 결과
에이전트 ①: WARN — "constraint_engine 필터 비례성 문제. 통과율 41%."
에이전트 ②: WARN — "모수 47건으로 DI 자체가 불안정."
에이전트 ③: PASS — "모수 부족으로 통계적 유의성 없음."
에이전트 ④: WARN — "필터 문제 + income 조건이 주 원인."
에이전트 ⑤: WARN — "계절 패턴 가능성. 지난 1월에도 유사 수치."

집계: WARN 4 / PASS 1
마이너리티: ③ (PASS)

## 지시
Round 1 전체 의견을 검토하고:
1. 다수의견(WARN)의 논거를 종합 정리하세요.
2. 마이너리티(③ PASS)의 근거가 타당한지 평가하세요.
   단, 마이너리티를 기각하더라도 원 의견은 보존됩니다.
3. 최종 종합 판정을 제시하세요.
```

3.2.4 합의 판정의 최종 분류

두 환경 모두 동일한 3단계로 최종 분류:

분류	AWS (3명)	온프렘 (5명)	처리
Consensus (만장일치)	3/3 일치	5/5 일치	통과 (PASS) 또는 확정 WARN. 추가 리뷰 불필요.
최우선 리뷰 (다수 이상)	2/3 이상 징후	3/5 이상 징후	즉시 담당자 리뷰. 다수 의견의 cause를 병합.
마이너리티 리포트 (소수 의견)	1/3 이상 징후	1 2/5 이상 징후	2순위 리뷰 리스트. 소수 의견 근거를 별도 기록. 케이스 스토어에 `consensus_type: "minority"` 태그.

3.2.5 적용 범위

합의 메커니즘은 모든 LLM 해석에 적용하지 않는다. 비용/시간 효율을 위해 고위험 판정에만 적용:

상황	AWS	온프렘	이유
체크리스트 WARN/FAIL 해석	O	O	잘못된 해석 → 잘못된 대응
변경 영향도 리뷰	O	O	누락 시 장애 위험
규제 적합성 판단	O	O	오판 시 규제 리스크
일상 대화 (담당자 질문)	–	N/A	저위험, 비용 불필요
유사 케이스 검색 해석	–	–	검색 자체는 결정론적

4 파이프라인 파트 분류 및 점검 체크리스트

에이전트가 파이프라인을 체계적으로 점검하려면, 먼저 파이프라인을 명확한 파트(부위)로 분할하고 각 파트에 대한 점검항목을 정의해야 한다.

4.1 6개 파트 분류

Figure 7: 파이프라인 6개 파트. 각 파트는 명확한 입력/출력 경계를 가진다.

파트	이름	범위 (스테이지)	주요 입력	주요 출력
P1	인제스천	IngestionRunner 전체	원천 데이터 (S3/DB)	정제된 Parquet + manifest
P2	피처 엔지니어링	Phase 0: Stage 1 9 (전처리 → 정규화 → 저장)	정제된 Parquet	features.parquet + labels.parquet + feature_schema.json
P3	학습 & 증류	교사 학습 → 분석 → 증류	학습 데이터 + config	ple_model.pt + LGBM 학생 + fidelity metrics
P4	서빙 & 추천	FeatureStore → Scorer → ConstraintEngine → TopKSelector	고객 ID + 모델 예측	추천 결과 (top-K items)
P5	추천사유 생성	TemplateEngine → LLM Rewrite → SelfChecker	추천 결과 + IG 기여도	사유 텍스트 + 검증 결과
P6	모니터링 & 거버넌스	드리프트/공정성/헤르딩/규제/ 감사 로그/거버넌스 리포트	전 파트의 측정값	진단 리포트 + 인시던트

4.2 에이전트별 파트 점검 체크리스트

각 파트에 대해 운영 에이전트와 감사 에이전트가 보는 관점이 다르다.

4.2.1 P1. 인제스천

#	에이전트	점검 항목	판정
1.1	Ops	도메인별 row count가 이전 배치 대비 ±20% 이내인가	PASS/WARN
1.2	Ops	validation 경고 (스키마 불일치, null 비율 초과)가 없는가	PASS/FAIL
1.3	Ops	인제스천 소요시간이 이전 대비 2× 미만인가	PASS/WARN
1.4	Ops	전체 도메인이 성공적으로 로드되었는가 (누락 도메인 없음)	PASS/FAIL
1.5	Audit	PII 컬럼이 모두 암호화/삭제 처리되었는가	PASS/FAIL
1.6	Audit	데이터 보존 기간 정책이 준수되는가	PASS/WARN
1.7	Audit	감사 로그에 인제스천 이벤트가 기록되었는가	PASS/FAIL

4.2.2 P2. 피처 엔지니어링

#	에이전트	점검 항목	판정
2.1	Ops	zero-variance 컬럼이 0개인가	PASS/WARN
2.2	Ops	NaN 비율이 전체 컬럼 기준 threshold 미만인가	PASS/WARN
2.3	Ops	피처 수가 예상 범위(config 기준) 내인가	PASS/WARN
2.4	Ops	정규화가 TRAIN split에서만 fit되었는가 (리키지 검증)	PASS/FAIL

#	에이전트	점검 항목	판정
2.5	Ops	leakage_report.json이 PASS인가	PASS/FAIL
2.6	Ops	스테이지별 소요시간이 정상 범위인가	PASS/WARN
2.7	Audit	피처 드리프트 (PSI)가 critical 미만인가	PASS/WARN/FAIL
2.8	Audit	멱법칙 피처가 올바르게 log1p 처리되었는가	PASS/WARN
2.9	Audit	데이터 계보 — 피처→원천 추적이 가능한가 (미매핑 < 5%)	PASS/WARN

4.2.3 P3. 학습 & 증류

#	에이전트	점검 항목	판정
3.1	Ops	전체 태스크의 val metric이 수렴했는가 (최근 3 epoch 변동 < 1%)	PASS/WARN
3.2	Ops	NaN/Inf loss가 발생하지 않았는가	PASS/FAIL
3.3	Ops	grad norm이 clip 임계값의 10× 미만인가	PASS/WARN
3.4	Ops	증류 fidelity gap이 태스크별 5% 미만인가	PASS/WARN
3.5	Ops	GPU 메모리 사용률이 OOM 위험 없는 범위인가	PASS/WARN
3.6	Ops	학습 비용이 예산 가드 이내인가	PASS/WARN
3.7	Audit	리트레이닝 전후 공정성 지표가 악화되지 않았는가	PASS/WARN/FAIL
3.8	Audit	증류 후 설명 재료 피처가 보존되었는가 (IG top-K 중복률)	PASS/WARN
3.9	Audit	실험 파라미터가 감사 로그에 기록되었는가	PASS/FAIL

4.2.4 P4. 서빙 & 추천

#	에이전트	점검 항목	판정
4.1	Ops	feature store health_check가 정상인가	PASS/FAIL
4.2	Ops	추천 응답 p95 latency가 SLA 이내인가	PASS/WARN/FAIL
4.3	Ops	filter 통과율이 정상 범위인가 (너무 낮으면 후보 부족)	PASS/WARN
4.4	Ops	kill switch 상태가 정상(비활성)인가	PASS/FAIL
4.5	Ops	A/B 테스트 variant 할당이 균등한가	PASS/WARN
4.6	Audit	단일 보호속성별 DI/SPD/EOD가 임계값 이내인가	PASS/WARN/FAIL
4.7	Audit	교차 보호속성 DI가 임계값 이내인가	PASS/WARN/FAIL
4.8	Audit	추천 집중도 (HHI/Gini)가 herding 임계값 미만인가	PASS/WARN
4.9	Audit	편향 Stage Attribution — 어느 단계에서 악화되는가	정보
4.10	Audit	추천 결과가 감사 아카이브에 기록되었는가	PASS/FAIL

4.2.5 P5. 추천사유 생성

#	에이전트	점검 항목	판정
5.1	Ops	사유 생성 latency가 전체 응답 SLA를 초과하지 않는가	PASS/WARN
5.2	Ops	L2a SQS 큐 깊이가 정상 범위인가 (백로그 없음)	PASS/WARN
5.3	Ops	DynamoDB reason_cache hit rate가 정상인가	PASS/WARN
5.4	Audit	Tier 1: SelfChecker pass rate가 임계값(95%) 이상인가	PASS/WARN
5.5	Audit	Tier 1: reject/revise 비율 추이가 악화되지 않는가	PASS/WARN
5.6	Audit	Tier 2: 품질 점수 (faithfulness, grounding) 추이	PASS/WARN
5.7	Audit	Tier 2: cross-method 일관성 (SHAP vs IG)이 유지되는가	PASS/WARN
5.8	Audit	Tier 3: 전문가 리뷰 대기 건수가 적정 범위인가	PASS/WARN
5.9	Audit	사유에 금지 패턴 (부적절 조언, prompt injection) 없는가	PASS/FAIL

#	에이전트	점검 항목	판정
5.10	Audit	AI 공시 문구가 포함되었는가	PASS/FAIL

4.2.6 P6. 모니터링 & 거버넌스

#	에이전트	점검 항목	판정
6.1	Ops	감사 로그 해시 체인이 무결한가 (verify_chain)	PASS/FAIL
6.2	Ops	인시던트 알림 (SNS)이 정상 작동하는가	PASS/FAIL
6.3	Ops	에이전트 자체가 스케줄대로 실행되었는가 (watchdog)	PASS/FAIL
6.4	Audit	국내 규제 20항목 중 critical failure가 없는가	PASS/FAIL
6.5	Audit	EU AI Act 17항목 compliance rate가 목표 이상인가	PASS/WARN
6.6	Audit	FRIA 종합 리스크가 HIGH 미만인가	PASS/WARN/FAIL
6.7	Audit	거버넌스 리포트가 주기에 맞게 생성되었는가	PASS/WARN
6.8	Audit	감사 패키지가 외부 제출 요건을 충족하는가	PASS/WARN

체크리스트 운영 방식
체크리스트는 YAML config로 관리하여 항목 추가/수정/비활성화가 가능하게 한다. 룰 엔진이 자동 판정하고, WARN/FAIL인 항목만 Sonnet 대화 인터페이스에 전달하여 담당자와 해석/대응을 논의한다. 체크리스트 전체 결과는 거버넌스 리포트의 부록으로 첨부된다.

5 운영 에이전트 (Ops Agent)

역할: 파이프라인이 "잘 돌아가고 있는가" — 성능, 안정성, 비용.

5.1 관찰 지점 — 7개 체크포인트

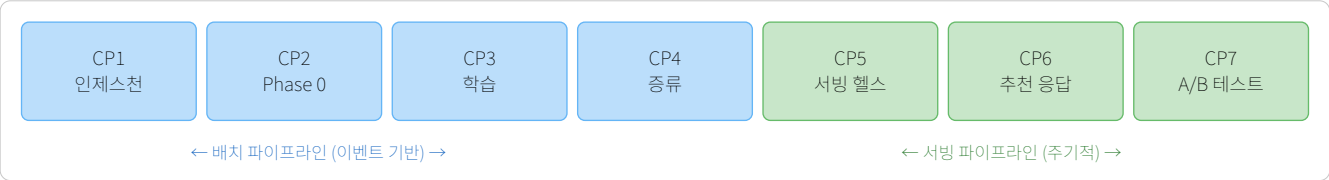


Figure 8: 운영 에이전트의 7개 체크포인트. 파란색은 배치 파이프라인, 초록색은 서빙 파이프라인.

#	체크포인트	수집 소스	측정 항목
CP1	인제스천 완료	`IngestionRunner .generate_manifest()`	도메인별 row count 변동률, PII 암호화 누락, validation 경고
CP2	Phase 0 완료	`pipeline_state.json` + `feature_stats.json`	스테이지별 소요시간, zero-variance 컬럼 수, NaN 비율 분포
CP3	학습 완료	`ExperimentTracker` (metrics.jsonl)	loss 수렴 여부, grad norm 이상, epoch별 val metric 추이
CP4	증류 완료	`DistillationValidator` fidelity 결과	태스크별 teacher-student AUC gap, fidelity 임계값 위반
CP5	서빙 헬스	`FeatureStore .health_check()` + CW	feature store 응답시간, 레코드 수 정합성, kill switch 상태
CP6	추천 응답	`audit_archiver` (Parquet traces)	p50/p95 latency, filter 통과율, top-K 다양성 지표

#	체크포인트	수집 소스	측정 항목
CP7	A/B 테스트	`ABTestManager` CloudWatch metrics	variant별 CTR/CVR, significance test, auto-promote 판단

5.2 진단 로직

5.2.1 시계열 이상 탐지

각 체크포인트의 측정값을 이전 실행과 비교하여 이상을 탐지한다:

- 인제스천 row count ±20% → "데이터 소스 이상 의심"
- Phase 0 소요시간 2× 이상 → "데이터 볼륨 급증 또는 generator 병목"
- 서빙 p95 latency > SLA (예: 200ms) → "feature store 또는 reason 생성 병목"

5.2.2 연쇄 영향 분석 (Cross-Checkpoint Correlation)

운영 에이전트의 핵심 가치는 단일 지표 알림이 아니라 체크포인트 간 상관관계 분석에 있다.

Figure 9: 연쇄 영향 분석 패턴. 두 체크포인트의 이상을 조합하여 원인을 추정.

5.2.3 비용 감시

- SageMaker billable time vs 예산 가드 (`pipeline.yaml ablation.budget_limit`)
- Spot 중단율 추적 → 연속 중단 시 on-demand 전환 권고
- Phase 0에 GPU 인스턴스 오사용 감지

5.3 리포트 형식

운영 리포트의 핵심은 finding + likely_cause + suggested_action 세트이다. 단순 알림이 아니라 체크포인트 간 상관관계를 분석해서 원인 추정까지 제공한다.

```yaml
ops_report:
  generated_at: "2026-04-10T09:00:00Z"
  period: "daily"
  status: YELLOW   # GREEN / YELLOW / RED

  attention_required:
    - checkpoint: CP3
      severity: WARNING
      finding: "churn_signal val_auc 3일 연속 하락 (0.82→0.79→0.76)"
      likely_cause: "CP2에서 tenure_months NaN 비율 12%→23% 증가"
      suggested_action: "인제스천 customer_master tenure 필드 품질 확인"

  all_checkpoints:
    CP1: {status: GREEN, rows: 941132, delta: "+0.2%"}
    CP2: {status: GREEN, duration: "4m32s", zero_var: 0}
    CP3: {status: YELLOW, detail: "1/14 tasks degrading"}
    CP4: {status: GREEN, max_fidelity_gap: "2.1%"}
    CP5: {status: GREEN, latency_p50: "12ms"}
```

```
  CP6: {status: YELLOW, detail: "p95 trending up"}
  CP7: {status: GREEN, detail: "no active experiment"}
```

5.4 실행 주기

체크포인트	트리거	주기
CP1 CP4	이벤트 기반 (각 스테이지 완료 시)	배치당 1회
CP5 서빙 헬스	주기적	5분
CP6 추천 응답	주기적 (1시간 집계)	상시
CP7 A/B 테스트	주기적	일 1회

6 감사 에이전트 (Audit Agent)

역할: 파이프라인이 "규정을 준수하고 있는가" — 공정성, 설명가능성, 추천사유 품질.

6.1 관찰 지점 — 5개 관점

#	관점	수집 소스	측정 항목
AV1	공정성	`FairnessMonitor`	보호속성별 DI/SPD/EOD, 위반 추이
AV2	집중도	`HerdingDetector`	HHI/Gini/Entropy, 태스크별 기여도 편중
AV3	추천사유 품질	`SelfChecker` + `XAIQualityEvaluator`	사유 통과율, faithfulness, stability, 샘플 심층검토
AV4	규제 적합성	`RegulatoryComplianceChecker` + `EUAIActMapper` + `FRIAEvaluator`	국내 20항목 + EU AI Act 17항목 준수율
AV5	데이터 계보	`DataLineageTracker`	추천→피처→원천 추적 가능 여부, 미매핑 피처 비율

6.2 추천사유 품질 검증 전략 (AV3 상세)

전수조사가 불가능하므로 3-Tier 샘플링 전략을 사용한다.

Tier 1 전수 자동검증 (100%)	• SelfChecker 통과율 모니터링 (reject / revise / pass 비율 추이) • 금지 패턴 탐지 (compliance_rules 위반) • prompt injection 탐지 기존 SelfChecker가 이미 전수 실행 — 집계/추이 분석만 추가
Tier 2 통계적 샘플링 (일 1 5%)	• 층화추출: 태스크유형 × 고객세그먼트 × 사유레이어(L1/L2a/L2b) 조합별 균등 • XAI 품질 평가 (faithfulness, stability, comprehensibility) • Grounding 검증: 사유 텍스트 ↔ IG top-K 피처 일치율 • Cross-method 일관성: SHAP vs IG 상위 피처 rank correlation
Tier 3 전문가 리뷰 (월 50 100건)	• Tier 2 경계선 사례 (confidence 0.6 0.8) 우선 추출 • 고위험 세그먼트 과표집 (elderly, low-income 등) • `human_review_flagged=True`인 L2b 결과 전수 포함 • 리뷰 결과 → Tier 1/2 규칙 업데이트 피드백 루프

Listing 1: 3-Tier 추천사유 품질 검증 전략. Tier가 올라갈수록 심층적이고 비용이 높다.

6.2.1 층화추출 설계 (Tier 2)

축	스트라텀	수
태스크 유형	binary, multiclass, regression	3
고객 세그먼트	mass, affluent, vip	3
사유 레이어	L1 (template), L2a (LLM rewrite), L2b (validated)	3
합계: 27개 스트라텀 × 각 10 20건		270 540건/일

6.2.2 우선 추출 조건

경계선 사례와 고위험 그룹을 과표집(oversampling)하여 검증 효율을 높인다:

조건	가중치	근거
`selfcheck_confidence ∈ [0.6, 0.8)`	3×	자동검증 경계선 — 오탐/미탐 집중 구간
고객 세그먼트 ∈ {elderly, low_income}	2×	보호계층 — 규제 감시 대상
`reason_layer = L2b ∧ human_review_flagged`	전수	L2b 플래그 건 — 자동검증이 불확실

6.2.3 품질 점수 체계

$$Q_{\text{reason}} = 0.30 \cdot F_{\text{faithfulness}} + 0.25 \cdot G_{\text{grounding}} + 0.25 \cdot C_{\text{compliance}} + 0.20 \cdot R_{\text{readability}}$$

차원	측정 방법	소스
Faithfulness	IG attribution과 perturbation 결과의 상관	`XAIQualityEvaluator`
Grounding	사유 텍스트에 언급된 피처가 IG top-K에 포함되는 비율	`ReverseMapper` + IG
Compliance	SelfChecker 통과 + 금지패턴 미검출	`SelfChecker`
Readability	문장 길이, 전문용어 비율, 모호한 표현 비율	규칙 기반

6.2.4 피드백 루프

전문가 리뷰 결과(Tier 3)는 자동검증 규칙(Tier 1/2)을 지속적으로 보정한다:

- 전문가 reject인데 Tier 1/2에서 pass → 새 `compliance_rule` 추가
- 전문가 accept인데 Tier 1/2에서 revise → 과도한 규칙 완화 검토
- 전문가 간 일치율(inter-rater agreement) 추적 → 규칙 신뢰도 지표

6.3 편향 감지 심화 (AV1 상세)

운영 에이전트가 보지 않는 세 가지 심층 분석을 감사 에이전트가 담당한다.

6.3.1 교차 보호속성 분석 (Intersectional Fairness)

단일 속성은 통과하지만 교차(intersection)에서 위반이 발생하는 케이스를 탐지한다.

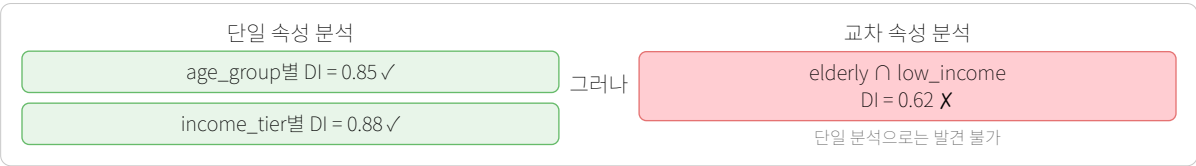


Figure 10: 교차 보호속성 분석. 단일 속성은 통과하지만 교차에서 위반이 발생하는 사례.

분석 대상 교차 조합 (config 정의):

- `age_group × income_tier` — 고령 저소득층 보호

- `gender × region_type` — 지역별 성별 격차
- `life_stage × income_tier` — 생애주기별 소득 격차

6.3.2 편향 발생 단계 분리 (Stage Attribution)
편향이 파이프라인의 어느 단계에서 발생/증폭되는지 분리 진단한다.

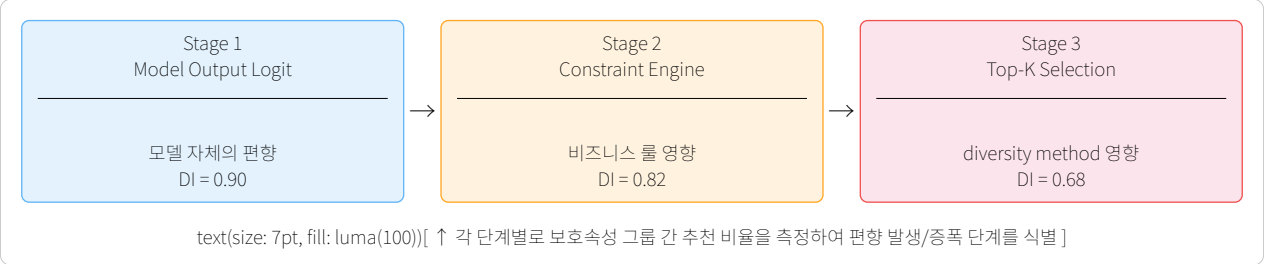
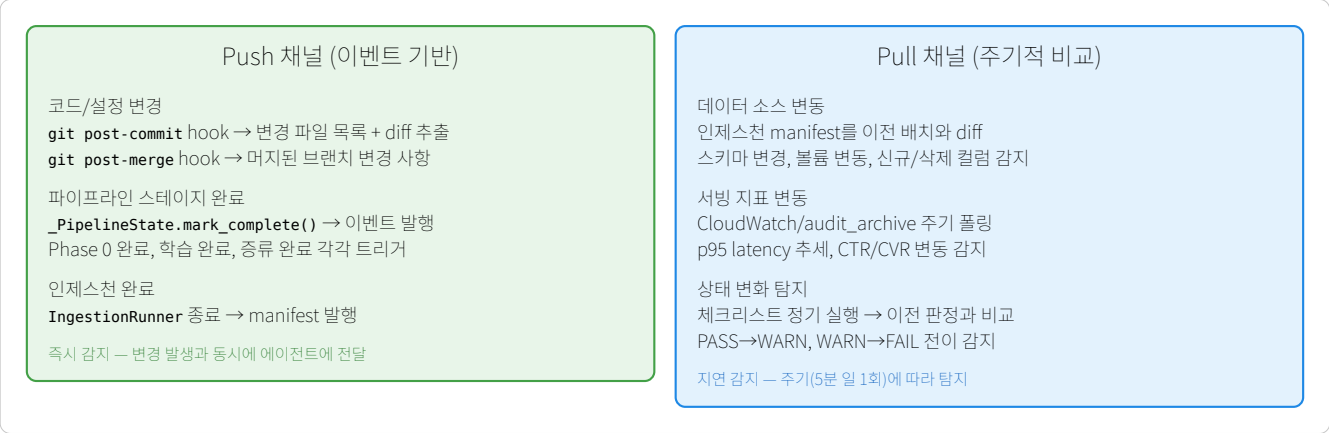


Figure 11: 편향 단계 분리 (Stage Attribution). 어느 단계에서 편향이 발생하고 증폭되는지 진단.

6.3.3 시계열 편향 추이
- 공정성 지표의 추세 (악화 중인가, 개선 중인가)
- 리트레이닝 전후 편향 변화 (의도치 않은 악화 감지)
- 계절성 패턴 (특정 시기에 반복되는 편향)

6.4 변경 영향도 리뷰 (Impact Review)
코드, 설정, 모델, 데이터 소스 등의 변경이 발생했을 때, 파이프라인 전반에 미치는 영향을 추론하고 담당자와 논의하는 기능. 이 기능이 LLM(Sonnet)이 필요한 핵심 이유이다.

6.4.1 변경 감지 메커니즘
영향도를 리뷰하려면 먼저 변경이 발생했다는 사실을 감지해야 한다. 두 가지 채널을 사용한다:



Listing 2: 변경 감지 이중 채널. Push는 즉시, Pull은 주기적으로 변경을 감지.

6.4.1.1 변경 유형별 감지 경로

변경 유형	채널	감지 소스	에이전트에 전달되는 정보
코드 변경	Push	`git post-commit` hook	변경 파일 경로, diff, 영향 받는 파이프라인 파트(P1 P6)
설정 변경	Push	`git post-commit` hook (YAML 파일 감지)	변경된 config 키, 이전값 → 신규값, 영향 받는 파트

변경 유형	채널	감지 소스	에이전트에 전달되는 정보
모델 변경	Push	`_PipelineState .mark_complete("stage_train")`	학습 완료 시각, val metric, 이전 모델 대비 변동
증류 변경	Push	`_PipelineState .mark_complete("stage_distill")`	태스크별 fidelity, 이전 증류 대비 변동
데이터 소스 변경	Pull	인제스천 manifest diff	도메인별 row count 변동, 스키마 변경, 신규/삭제 컬럼
서빙 지표 변동	Pull	CloudWatch / audit_archive	latency 추세, filter 통과율 변동, CTR 변동
규제 변경	수동	담당자 입력	변경된 규제 항목, 신규 요구사항

6.4.1.2 구현: 변경 이벤트 표준 포맷

모든 변경 감지는 동일한 이벤트 포맷으로 에이전트에 전달된다:

```
{
  "event_type": "change_detected",
  "change_type": "code",
  "source": "git_post_commit",
  "timestamp": "2026-04-10T14:30:00Z",
  "details": {
    "commit_hash": "a1b2c3d",
    "changed_files": [
      "core/recommendation/constraint_engine.py"
    ],
    "affected_parts": ["P4", "P5"],
    "diff_summary": "+42 -8 lines in 1 file"
  }
}
```

6.4.1.3 온프렘 vs AWS: 변경사항 관리 방식의 구조적 차이

온프렘과 AWS는 변경사항의 추적 가능성(traceability) 수준이 근본적으로 다르다. AWS는 CloudTrail, S3 버전관리, Model Registry 등이 자동으로 이력을 남기지만, 온프렘은 의도적으로 기록하지 않으면 누락된다.

변경 대상	온프렘	AWS
코드/설정	로컬 git → 사내 git 서버 push `post-commit` hook → 파일 이벤트	GitHub/CodeCommit → CI/CD `post-commit` hook + CloudTrail API 추적
모델	로컬 학습 → 파일시스템 체크포인트 `pipeline_state.json`이 로컬 디스크 버전 관리는 수동 (디렉토리명/타임스탬프)	SageMaker Training Job → S3 아티팩트 Model Registry로 자동 버전 관리 학습 Job 이력이 CloudTrail에 기록
데이터	DuckDB 파일 기반 인제스천 manifest 로컬 저장 이전 manifest와 수동 비교	S3 Parquet + S3 버전관리 자동 활성 S3 이벤트 알림으로 변경 즉시 감지 Glue Data Catalog로 스키마 추적
배포	Docker 컨테이너 직접 배포 배포 이력은 사내 CI/CD에만	SageMaker Endpoint 업데이트 CloudTrail로 모든 API 추적 Blue/Green 배포 이력 자동 기록
규제 설정	config YAML 수정 → git hook 양쪽 동일	config YAML 수정 → git hook 양쪽 동일

이 차이가 에이전트의 변경 감지 방식에 직접 영향을 준다:

채널	온프렘	AWS
git hook	`post-commit` → 룰 엔진에 이벤트 전달 → 영향 받는 파트 체크리스트 재실행	같은 방식 + Sonnet에 diff 전달 → 영향도 대화
파이프라인 이벤트	`mark_complete()` → JSON 파일 기록 → 룰 엔진이 주기적 확인 (Pull)	EventBridge 이벤트 발행 → Lambda 즉시 트리거 (Push)
모델 버전	디렉토리명/타임스탬프 비교 → 수동 또는 스크립트 감지	Model Registry 이벤트 → 자동 감지 + 이전 버전 메타데이터 즉시 비교
데이터 변경	manifest 파일 생성 → 이전 파일과 diff	S3 이벤트 알림 + Glue Catalog diff → 스키마/볼륨 변경 즉시 감지
서빙 지표	audit_archive Parquet 주기 스캔	CloudWatch Alarm → SNS → 임계값 초과 즉시 트리거
배포 변경	Docker 컨테이너 교체 감지 필요 → watchdog 스크립트	CloudTrail `UpdateEndpoint` API → 자동 감지

온프렘의 보완 전략
온프렘에서 AWS 수준의 자동 추적이 불가능한 영역(모델 버전, 배포 이력)은 에이전트의 Action 도구로 보완한다:
(1) 모델 학습/증류 완료 시 `log_audit_event`를 자동 호출하여 버전/메트릭 기록
(2) 배포 스크립트에 `log_audit_event` 호출을 내장하여 배포 이력 기록
(3) 이렇게 기록된 감사 로그가 곧 변경 이력이 되어, 에이전트의 Pull 채널에서 읽힌다
결과적으로 온프렘도 "누가 언제 뭘 바꿨는지"를 추적할 수 있지만, AWS처럼 자동이 아니라 규약 기반이다.

변경 감지 → 영향도 리뷰 연결
온프렘: 변경 감지 시 영향 받는 파트의 체크리스트만 재실행. 코드 변경이 P4(서빙)에 영향이면 P4 체크리스트 7개 항목을 즉시 재판정. 결과가 이전과 달라지면(PASS→WARN 등) 정형 리포트에 포함.
AWS: 동일한 재판정 + Sonnet이 diff를 읽고 "이 변경이 왜 이 파트에 영향을 주는지" 맥락을 설명. 담당자가 추가 도구 호출을 요청하며 대화 가능.

6.4.2 리뷰 대상 변경 유형

변경 유형	예시	잠재 영향 범위
코드 변경	constraint_engine 필터 추가, scorer 가중치 수정	공정성(DI), 추천 다양성, 사유 품질, 규제 준수
설정 변경	pipeline.yaml 태스크 가중치, feature_groups.yaml 그룹 변경	모델 성능, 피처 라우팅, 증류 fidelity
모델 변경	리트레이닝, 증류 재실행, 학생 모델 교체	서빙 지표, 사유 품질(IG 변동), 편향 프로파일
데이터 소스 변경	인제스천 도메인 스키마 변경, 신규 컬럼 추가/삭제	피처 엔지니어링, 정규화, 모델 입력 차원
규제 변경	금감원 가이드라인 개정, EU AI Act 시행세칙 변경	compliance_rules 업데이트, 감사 항목 추가

6.4.3 Sonnet이 수행하는 영향도 추론

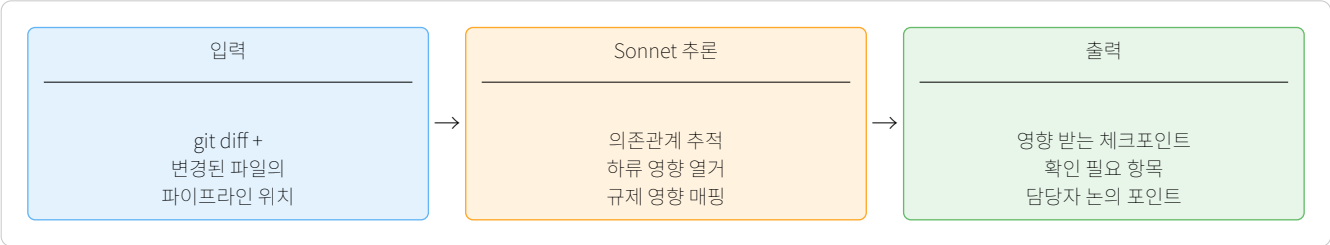


Figure 12: 변경 영향도 리뷰 흐름. Sonnet이 코드를 읽고 파이프라인 의존관계를 추론.

6.4.4 대화 예시

```
담당자: "constraint_engine에 신규 필터 하나 추가했는데 영향도 봐줘"

Ops 에이전트:
  "constraint_engine은 CP6 추천 응답 체크포인트에 직접 영향합니다.
   필터가 추가되면 filter 통과율이 하락할 수 있고,
   top-K 후보 풀이 줄어서 다양성 지표에도 영향 가능합니다.
   CP6 p95 latency도 필터 연산만큼 증가합니다."

Audit 에이전트:
  "공정성 관점에서 이 필터가 특정 보호속성 그룹에
   불비례적으로 적용되는지 확인이 필요합니다.
   현재 elderly ∩ low_income 그룹의 DI가 0.68로 이미 위반 상태인데,
   신규 필터가 이 그룹의 후보를 추가로 줄이면 악화됩니다.
   배포 전에 해당 그룹 필터 통과율을 시뮬레이션할 것을 권고합니다."

담당자: "그러면 이 필터를 해당 그룹에는 적용 안 하면?"

Audit 에이전트:
  "그건 positive action으로 정당화 가능하지만,
   금소법 적합성 원칙과 충돌할 수 있습니다 —
   필터가 적합성 기반이면 그룹 예외는 오히려 규제 위반입니다.
   필터의 목적이 적합성인지 비즈니스 룰인지에 따라 판단이 달라집니다."
```

6.5 감사 리포트 형식

감사 리포트의 핵심은 focus_areas이다: “어디를 집중적으로 봐야 하는가”를 우선순위 순으로 제시한다.

```yaml
audit_report:
  generated_at: "2026-04-10T09:00:00Z"
  period: "weekly"
  risk_level: MEDIUM

  focus_areas:
    - area: "추천사유 품질"
      priority: HIGH
      finding: |
        L2b 사유 중 grounding < 0.7 비율이 18%→24%로 증가.
        nba_primary 태스크에서 IG top-3 불일치 집중 발생.
      evidence:
        tier2_sample_size: 540
        grounding_below_threshold: 129
        worst_task: "nba_primary"
      recommended_review: |
        nba_primary의 L2b 사유 중 grounding < 0.5인 32건을
```

```yaml
      Tier 3 전문가 리뷰 대상에 추가 권고.

  - area: "교차속성 공정성"
    priority: MEDIUM
    finding: |
      elderly ∩ low_income 그룹의 DI = 0.68 (기준 0.80).
      단일 속성은 모두 통과, 교차에서 위반.
    evidence:
      subgroup: "elderly ∩ low_income"
      pipeline_stage_attribution: "constraint_engine에서 악화"
    recommended_review: |
      constraint_engine eligibility 필터의 비례성 점검 권고.

regulatory_summary:
  domestic:
    financial_consumer_protection: {pass: 4, fail: 1}
    personal_info_protection: {pass: 6, fail: 0}
    ai_basic_act: {pass: 8, fail: 1}
  eu_ai_act:
    risk_classification: "HIGH"
    compliance_rate: 0.82

reason_quality_dashboard:
  tier1_auto:
    total_checked: 142000
    pass_rate: 0.96
  tier2_sample:
    sample_size: 540
    avg_quality_score: 0.74
    trend: "grounding declining"
  tier3_expert:
    pending_review: 82
    agreement_with_auto: 0.89
```


6.6 실행 주기

관점	트리거	주기
AV1 공정성	주기적 + Ops 트리거	일 1회 (drift 시 즉시)
AV2 집중도	주기적	일 1회
AV3 추천사유	Tier 1: 실시간, Tier 2: 일 1회, Tier 3: 월 1회	혼합
AV4 규제 적합성	주기적	주 1회 (분기 1회 전체)
AV5 데이터 계보	이벤트 기반 (모델 변경 시)	변경당 1회

7 두 에이전트 간 관계

7.1 상호 트리거

두 에이전트는 독립적으로 실행되지만, 특정 조건에서 상대 에이전트에 트리거를 발행한다.

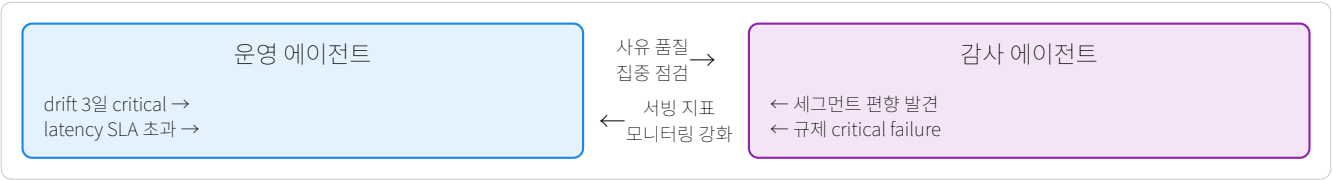


Figure 13: 두 에이전트 간 상호 트리거 관계.

조건	발신	수신	트리거 내용
drift PSI critical 3일 연속	Ops	Audit	모델 성능 저하 구간의 추천사유 품질 집중 점검
특정 세그먼트 편향 발견	Audit	Ops	해당 세그먼트 서빙 지표 모니터링 강화
서빙 latency SLA 초과	Ops	Audit	latency 문제 구간의 사유 생성 skip 여부 확인
규제 critical failure	Audit	Ops	해당 규제 관련 파이프라인 스테이지 상태 즉시 확인

7.2 거버넌스 리포트 통합

기존 `GovernanceReportGenerator`의 9개 섹션에 두 에이전트 결과를 공급:

거버넌스 리포트 섹션	데이터 소스
fairness_summary	Audit AV1
drift_summary	Ops CP2
incident_summary	양쪽 공유 (IncidentReporter)
model_changes	Ops CP3, CP4
kill_switch_history	Ops CP5
recommendation_quality	Audit AV3
herding_summary	Audit AV2
audit_summary	Audit AV4
executive_summary	양쪽 attention_required / focus_areas 통합

8 기존 컴포넌트 재사용 매핑

8.1 재사용 (기존)

컴포넌트	에이전트	역할
`DriftDetector` / `PSICalculator`	Ops	피처 드리프트 감지
`FairnessMonitor`	Audit	단일 속성 공정성
`HerdingDetector`	Audit	추천 집중도
`SelfChecker`	Audit	사유 자동검증 (Tier 1)
`XAIQualityEvaluator`	Audit	설명 품질 평가 (Tier 2)
`RegulatoryComplianceChecker`	Audit	국내 규제 20항목
`EUAIActMapper`	Audit	EU AI Act 17항목
`FRIAEvaluator`	Audit	리스크 영향 평가
`DataLineageTracker`	Audit	데이터 계보
`IncidentReporter`	양쪽	긴급 에스컬레이션

컴포넌트	에이전트	역할
`GovernanceReportGenerator`	양쪽	월간 통합 리포트
`AuditPackageBuilder`	Audit	외부 감사 패키지

8.2 신규 개발 필요

컴포넌트	에이전트	설명
`OpsCollector`	Ops	7개 체크포인트 측정값 수집기
`OpsDiagnoser`	Ops	연쇄 영향 분석 (cross-checkpoint)
`OpsReporter`	Ops	리포트 생성 + 전달 (Slack/Email/SNS)
`StratifiedReasonSampler`	Audit	Tier 2 층화추출 + 우선순위 샘플링
`GroundingValidator`	Audit	사유 텍스트 ↔ IG top-K 일치율 검증
`IntersectionalFairnessAnalyzer`	Audit	교차 보호속성 분석
`BiasStageAttributor`	Audit	편향 발생/증폭 단계 분리
`AuditDiagnoser`	Audit	focus_areas 생성 + 우선순위 판단
`AuditReporter`	Audit	리포트 생성 + 전달
`AgentEventBridge`	양쪽	상호 트리거 + GovernanceReport 연동
`DiagnosticCaseStore`	양쪽	LanceDB 기반 진단 케이스 저장/검색/통계
`ConsensusArbiter`	양쪽	3-에이전트 합의 판정 + 마이너리티 분류 (AWS: 독립 투표 / 온프렘: 2-Round 하이브리드)
`ChangeDetector`	양쪽	변경 감지 Push/Pull 채널 + 표준 이벤트 포맷 발행
`ToolRegistry`	양쪽	도구 정의(JSON Schema) 관리 + 온프렘 직접 호출 / AWS Bedrock Tool Use 연동
`SendNotification`	양쪽	Slack/Email/SNS 리포트 전달 (Action 도구)

9 도구 호출 체계 (Tool Calling)

에이전트가 파이프라인을 점검하려면 각 컴포넌트를 도구(tool)로 호출할 수 있어야 한다. 온프렘에서는 Python 직접 호출, AWS에서는 Bedrock Tool Use 포맷으로 동일한 도구를 노출한다.

9.1 설계 원칙

Query/Action 분리	읽기 도구(Query)와 쓰기 도구(Action)를 명확히 구분. 감사 에이전트가 실수로 상태를 변경하는 것을 구조적으로 방지. Action 도구는 명시적 승인 후에만 실행.
단일 인터페이스	온프렘: `ToolRegistry.call("tool_name", params)` → Python 직접 호출. AWS: Bedrock `tool_use` 블록 → Lambda/직접 호출. 도구 정의(JSON Schema)는 동일.
최소 권한	각 에이전트는 필요한 도구만 접근. Ops는 서빙 헬스/메트릭 도구, Audit는 공정성/규제/사유 품질 도구. 공통 도구(파일 읽기, 인시던트)만 공유.

9.2 도구 카탈로그 — 4개 범주

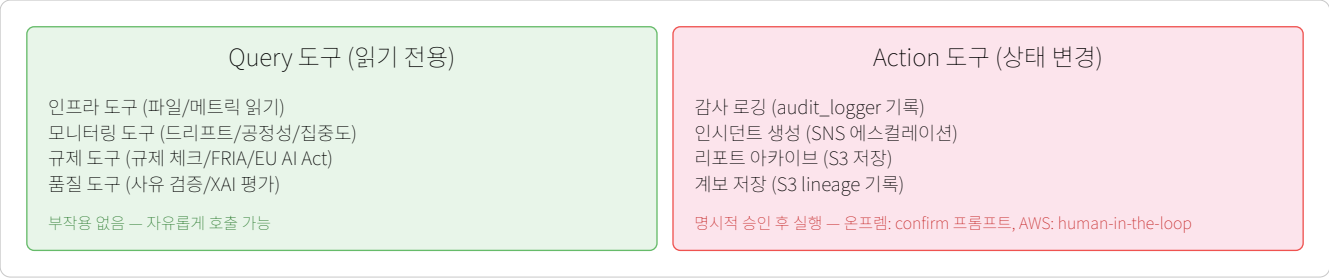


Figure 14: 도구 카탈로그 4개 범주. Query와 Action의 명확한 분리.

9.2.1 범주 1: 인프라 도구 (Query)

파이프라인 산출물과 인프라 상태를 읽는 도구. 기존 컴포넌트와 무관하게 신규 개발 필요.

도구 이름	설명	Ops	Audit
`read_pipeline_state`	pipeline_state.json 읽기 — 스테이지별 완료 상태, 소요시간, artifact 메타데이터	O	O
`read_feature_stats`	feature_stats.json 읽기 — 피처별 mean/std/null비율/zero-variance 여부	O	O
`read_experiment_metrics`	metrics.jsonl 읽기 — epoch별 loss, val_auc, grad_norm 등 학습 추이	O	–
`read_ingestion_manifest`	인제스천 manifest 읽기 — 도메인별 row count, PII 처리 현황, validation 상태	O	O
`read_leakage_report`	audit/leakage_report.json 읽기 — 리키지 검증 결과	O	–
`read_distillation_fidelity`	증류 결과에서 태스크별 teacher-student fidelity gap 추출	O	O
`query_cloudwatch_metrics`	CloudWatch 메트릭 조회 — p50/p95 latency, A/B variant별 CTR (AWS 전용)	O	–
`read_audit_archive`	audit_archiver Parquet에서 추천 결과 통계 집계 — filter 통과율, 다양성 지표	O	O
`read_git_diff`	git diff 읽기 — 변경된 파일, 라인 수, 영향 받는 파이프라인 파트 (AWS Bedrock 전용)	O	O
`read_checklist_config`	체크리스트 YAML config 읽기 — 현재 활성 점검 항목, 임계값 설정	O	O

9.2.2 범주 2: 모니터링 도구 (Query)

기존 `core/monitoring/` 컴포넌트를 도구로 래핑.

도구 이름	래핑 대상	Ops	Audit
`detect_drift`	`DriftDetector.detect_drift()` — 피처별 PSI, warning/critical 분류	O	O
`get_consecutive_drift_days`	`ConsecutiveDriftTracker.get_consecutive_critical_days()` — 연속 critical 일수	O	O
`evaluate_fairness`	`FairnessMonitor.evaluate_all_attributes()` — 보호속성별 DI/SPD/EOD	–	O
`detect_herding`	`HerdingDetector.detect_herding()` — HHI/Gini/Entropy, severity 판정	–	O
`detect_task_herding`	`HerdingDetector.detect_task_herding()` — 태스크별 기여도 편중	–	O
`check_feature_store_health`	`FeatureStore.health_check()` — 백엔드 상태, 레코드 수, 응답시간	O	–
`evaluate_data_quality`	`QualityGate.evaluate()` — 스키마/null/범위/드리프트/PII 검증	O	O

9.2.3 범주 3: 규제 · 품질 도구 (Query)

기존 `core/compliance/`, `core/monitoring/`, `core/recommendation/reason/` 컴포넌트 래핑.

도구 이름	래핑 대상	Ops	Audit
`run_regulatory_checks`	`RegulatoryComplianceChecker.run_all_checks()` — 국내 3법 20항목	–	O
`run_compliance_check`	`ComplianceChecker.run_full_check()` — 9항목 인프라 준수	–	O
`evaluate_eu_ai_act`	`EUAIActMapper.generate_report()` — EU AI Act 17개 조항 준수	–	O
`evaluate_fria`	`FRIAEvaluator.generate_report()` — 5차원 리스크 평가	–	O
`check_reason_quality`	`SelfChecker.check()` — 단일 사유 compliance/injection/factuality	–	O
`evaluate_xai_quality`	`XAIQualityEvaluator.evaluate_task()` — faithfulness/stability/comprehensibility	–	O
`check_explanation_consistency`	`XAIQualityEvaluator.check_explanation_consistency()` — SHAP vs IG rank 일관성	–	O
`trace_feature_lineage`	`DataLineageTracker.trace_features_batch()` — 피처→원천 추적	–	O
`generate_lineage_report`	`DataLineageTracker.generate_lineage_report()` — 배치 계보 리포트	–	O
`verify_audit_chain`	`AuditLogger.verify_chain()` — 감사 로그 해시 체인 무결성	O	O

9.2.4 범주 4: Action 도구 (상태 변경)

호출 시 S3, DynamoDB, SNS 등에 기록하는 도구. 명시적 승인 후 실행.

도구 이름	래핑 대상	Ops	Audit
`create_incident`	`IncidentReporter.create_incident()` — 인시던트 생성 + SNS 에스컬레이션	O	O
`log_audit_event`	`AuditLogger.log_operation()` — HMAC 서명 감사 로그 기록	O	O
`archive_governance_report`	`GovernanceReportGenerator.archive_report()` — S3 거버넌스 리포트 아카이브	–	O
`save_compliance_report`	`ComplianceChecker.save_report()` — S3 준수 보고서 저장	–	O
`save_lineage`	`DataLineageTracker.save_lineage()` — S3 계보 데이터 저장	–	O
`generate_governance_report`	`GovernanceReportGenerator.generate_report()` — 월간 거버넌스 리포트 생성	O	O
`send_notification`	Slack/Email/SNS로 리포트 전달 (신규 개발)	O	O

9.3 도구 호출 흐름

온프렘: Python 직접 호출

```python
# 룰 엔진이 체크리스트 순회
for item in checklist:
    result = registry.call(
        item.tool, item.params
    )
    verdict = judge(result, item.threshold)
```

ToolRegistry: Dict[str, Callable]
체크리스트 YAML → 도구 이름 + 파라미터 + 임계값
판정 결과 → 정형 리포트 템플릿

AWS: Bedrock Tool Use

```
{"type": "tool_use",
 "name": "detect_drift",
 "input": {"feature_columns":
   ["spend_*", "txn_*"]}}
```

온프렘 엔진 그대로 +
Sonnet이 대화 중 도구를 선택적 호출
"이 그룹의 DI를 직접 확인해볼게요"
→ `evaluate_fairness` 호출 → 결과 해석

Listing 3: 온프렘과 AWS의 도구 호출 방식. 동일한 도구 정의, 다른 호출 메커니즘.

9.3.1 도구 정의 형식 (JSON Schema)

모든 도구는 동일한 JSON Schema로 정의되어 온프렘/AWS에서 공유된다:

```
{
  "name": "evaluate_fairness",
  "description": "보호속성별 공정성 지표(DI/SPD/EOD) 평가",
  "category": "query",
  "agents": ["audit"],
  "parameters": {
    "type": "object",
    "properties": {
      "recommendations": {
        "type": "string",
        "description": "추천 결과 Parquet 경로 또는 날짜 범위"
      },
      "attributes": {
        "type": "array",
        "items": {"type": "string"},
        "description": "평가 대상 보호속성 목록"
      }
    },
    "required": ["recommendations"]
  },
  "returns": "FairnessMetrics (속성별 DI/SPD/EOD, 위반 목록)"
}
```

9.3.2 체크리스트 ↔ 도구 매핑

체크리스트의 각 항목은 하나 이상의 도구 호출로 자동 판정된다:

체크 항목	호출 도구	판정 로직
1.1 row count 변동	`read_ingestion_manifest`	`abs(current - previous) / previous < 0.20`
2.5 리키지 PASS	`read_leakage_report`	`result["passed"] == True`
3.4 fidelity gap	`read_distillation_fidelity`	`max(gap_per_task) < 0.05`
4.2 p95 latency	`query_cloudwatch_metrics` / `read_audit_archive`	`p95_ms < sla_threshold`
4.6 공정성 DI	`evaluate_fairness`	`all(di >= 0.80 for di in results)`
5.4 사유 pass rate	`read_audit_archive` (집계)	`pass_count / total >= 0.95`
6.1 해시 체인	`verify_audit_chain`	`result == True`

9.3.3 AWS 확장: Sonnet 대화 중 도구 호출

AWS 환경에서 Sonnet은 담당자와 대화하면서 필요할 때 도구를 직접 호출한다:

```
담당자: "elderly ∩ low_income 그룹이 실제로 얼마나 영향 받는지 봐줘"

Sonnet: [도구 호출: evaluate_fairness(
          recommendations="2026-04-09",
          attributes=["age_group", "income_tier"]
        )]

        결과를 보니 age_group=elderly 단독은 DI 0.85로 통과하지만,
        income_tier=low와 교차하면 DI 0.62입니다.
        전체 832건 중 해당 그룹은 47건이라 모수가 작아서
        필터 1개만 추가되어도 DI가 크게 흔들리는 구조입니다.

담당자: "constraint_engine 필터 통과율도 같이 비교해줘"
```

```
Sonnet: [도구 호출: read_audit_archive(
          date="2026-04-09",
          group_by=["age_group", "income_tier"],
          metric="filter_pass_rate"
        )]

        전체 평균 filter 통과율 67%인데,
        elderly ∩ low_income은 41%입니다.
        eligibility 필터에서 주로 걸리고 있네요.
```

이것이 온프렘 정형 리포트와의 핵심 차이이다: 담당자가 궁금한 것을 즉석에서 도구로 확인하며 대화할 수 있다.

9.4 도구 수량 요약

범주	Query	Action	Ops	Audit
인프라	10	0	9	7
모니터링	7	0	4	5
규제 · 품질	10	0	1	10
케이스 스토어	2	2	2	2
Action (기타)	0	7	4	6
합계	29	9	20	30

> 부작용 있는 Query 도구 처리
> `FairnessMonitor.evaluate_fairness()`와 `HerdingDetector.detect_herding()`은 Query이지만 `auto_incident=True`일 때 인시던트를 자동 생성하는 부작용이 있다. 에이전트 도구로 래핑할 때는 `auto_incident=False`로 고정하고, 인시던트 생성은 별도 Action 도구(`create_incident`)로만 수행하게 한다. 이렇게 해야 Query/Action 경계가 깨지지 않는다.

10 진단 케이스 스토어 (Diagnostic Case Store)

점검 리포트는 일회성 산출물이 아니다. 누적되면 운영 지식 베이스가 되어, 유사 케이스 참조, 통계 분석, 대응 효과 추적이 가능해진다.

기존 추천사유의 `ContextVectorStore`(LanceDB + numpy fallback)와 동일한 패턴을 적용한다.

10.1 개념

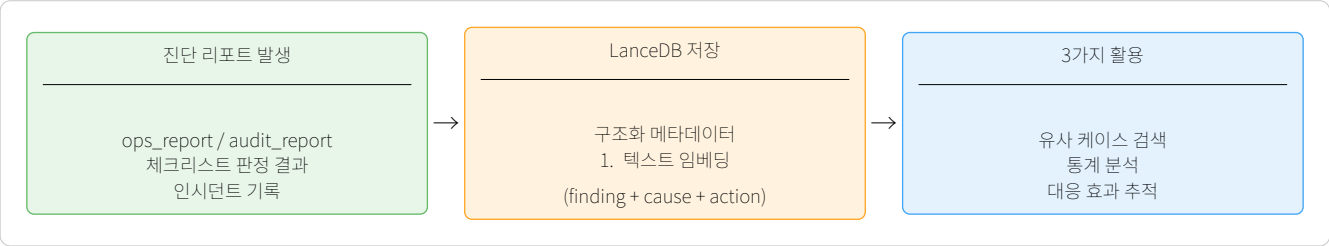


Figure 15: 진단 케이스 스토어 흐름. 리포트 → LanceDB → 유사 검색/통계/효과 추적.

10.2 케이스 스키마

각 진단 케이스는 구조화된 메타데이터와 텍스트 임베딩 두 가지로 저장된다.

```
{
  "case_id": "OPS-2026-04-10-001",
  "timestamp": "2026-04-10T09:00:00Z",
```

```
  "agent": "ops",
  "pipeline_part": "P3",
  "check_item": "3.1",
  "verdict": "WARN",
  "severity": "WARNING",

  "finding": "churn_signal val_auc 3일 연속 하락 (0.82→0.79→0.76)",
  "likely_cause": "CP2 tenure_months NaN 비율 12%→23% 증가",
  "suggested_action": "인제스천 customer_master tenure 필드 품질 확인",

  "metrics": {
    "val_auc": 0.76,
    "nan_ratio_tenure": 0.23,
    "drift_psi": 0.18
  },

  "consensus_type": "majority",
  "consensus_detail": {
    "round1_votes": {"WARN": 4, "PASS": 1},
    "minority_agents": ["③"],
    "minority_reasoning": "모수 47건으로 통계적 유의성 부족"
  },

  "resolution": null,
  "resolved_at": null,
  "post_resolution_verdict": null,

  "vector": [0.12, -0.34, ...]
}
```

핵심 필드:

- `pipeline_part` + `check_item`: 파이프라인 어디서 발생했는가 (필터링/집계용)
- `finding` + `likely_cause` + `suggested_action`: 텍스트 임베딩 대상 (유사 검색용)
- `metrics`: 수치 (통계 분석용)
- `consensus_type`: `"consensus"` / `"majority"` / `"minority"` / `null` (합의 분류)
- `consensus_detail`: Round 1 투표 결과, 마이너리티 에이전트 ID 및 근거
- `resolution` → `post_resolution_verdict`: 대응 후 효과 추적용

10.3 3가지 활용

10.3.1 활용 1: 유사 케이스 검색

새로운 이상이 감지되면, 과거에 유사한 패턴이 있었는지 검색한다.

```
현재 이상: "spend_* 피처 그룹 PSI 0.31, 3일 연속 critical"

유사 케이스 검색 (top-3):
┌──────────────────────────────────────────┐
│ #1 OPS-2026-02-15-003 (유사도: 0.91)      │
│ "spend_* PSI 0.28 → 인제스천 transaction 도메인  │
│  스키마 변경이 원인. 어댑터 매핑 수정으로 해결."  │
│ → 해결까지 2일, 해결 후 PSI 0.04로 정상화      │
├──────────────────────────────────────────┤
│ #2 OPS-2026-01-22-007 (유사도: 0.84)      │
│ "txn_count_* PSI 0.35 → 연말 거래량 급증 패턴.  │
│  계절적 요인으로 리트레이닝으로 해결."        │
│ → 해결까지 5일 (리트레이닝 대기)             │
└──────────────────────────────────────────┘
```

Listing 4: 유사 케이스 검색 예시. 과거 대응 이력과 해결 기간을 참조.

검색 방식:
- finding + likely_cause를 임베딩하여 코사인 유사도 검색
- `pipeline_part` 필터로 같은 파트의 케이스 우선
- 상위 3 5건 반환, 각 케이스의 resolution과 해결 소요시간 포함

10.3.2 활용 2: 통계 분석
구조화된 메타데이터로 운영 패턴을 분석한다. LanceDB의 SQL 호환 쿼리 또는 DuckDB 연동으로 집계.

분석 질문	쿼리 방법
P4 서빙 파트에서 가장 빈번한 WARN 유형은?	`WHERE pipeline_part='P4' AND verdict='WARN'` `GROUP BY check_item ORDER BY count DESC`
최근 3개월 공정성 위반 추이는?	`WHERE check_item LIKE '4.6%' OR '4.7%'` `GROUP BY week ORDER BY week`

평균 해결 소요시간이 가장 긴 파트는?	`WHERE resolved_at IS NOT NULL`
	`GROUP BY pipeline_part`
	`AVG(resolved_at - timestamp)`

인제스천 row count 변동이 모델 성능에 영향을 준 빈도는?	`WHERE check_item='1.1' AND verdict='WARN'` 발생 후 7일 내 `check_item='3.1' AND verdict IN ('WARN','FAIL')` 발생 비율
특정 suggested_action의 실제 효과는?	`WHERE suggested_action LIKE '%리트레이닝%'` 대응 후 `post_resolution_verdict` PASS 비율

10.3.3 활용 3: 대응 효과 추적 (Resolution Feedback Loop)
"이 대응이 실제로 효과가 있었는가"를 추적한다.

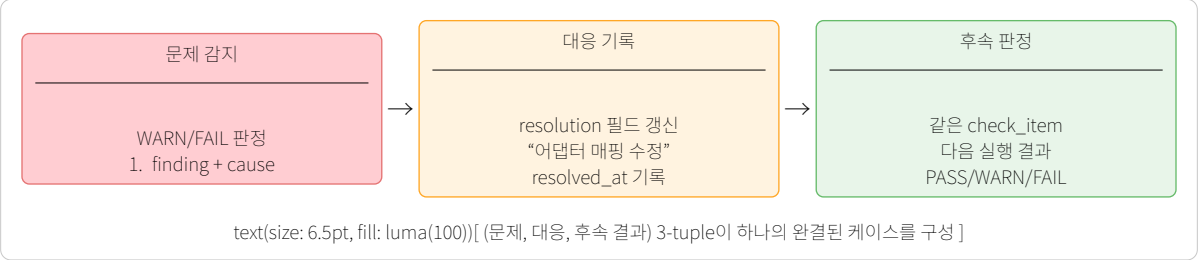


Figure 16: 대응 효과 추적. 문제 → 대응 → 후속 판정의 3-tuple로 케이스 완결.

이 피드백 루프의 효과:
- "리트레이닝으로 drift 해결" 같은 대응의 성공률을 정량화
- 비효과적 대응 패턴 식별 → suggested_action 룰 테이블 개선
- 거버넌스 리포트에 "대응 효과 분석" 섹션으로 포함

10.4 AWS 확장: Sonnet + 케이스 스토어 연동

AWS에서 Sonnet 대화 시, 케이스 스토어가 도구로 노출된다:

```
담당자: "지금 이 drift 패턴, 전에도 있었나?"

Sonnet: [도구 호출: search_similar_cases(
          finding="spend_* PSI 0.31, 3일 연속 critical",
          pipeline_part="P2",
          k=3
        )]

        과거 유사 케이스가 2건 있습니다.
```

```
2월 15일 건은 인제스천 스키마 변경이 원인이었고
어댑터 매핑 수정으로 2일 만에 해결했습니다.
1월 22일 건은 계절적 요인이라 리트레이닝이 필요했고
5일 걸렸습니다.
지금 상황은 2월 건과 더 유사해 보입니다 —
최근 인제스천 스키마 변경이 있었는지 확인해볼까요?
```

온프렘에서는 유사 케이스 검색 결과가 정형 리포트에 자동 첨부된다.

10.5 케이스 스토어 도구 (추가)

도구 카탈로그에 다음 3개 도구 추가:

도구 이름	설명	Query/Action	에이전트
`search_similar_cases`	finding 텍스트로 유사 케이스 벡터 검색 (top-K)	Query	양쪽
`get_case_statistics`	파트별/항목별/기간별 케이스 통계 집계	Query	양쪽
`save_case`	진단 결과를 케이스로 저장 (벡터 + 메타데이터)	Action	양쪽
`update_case_resolution`	케이스의 resolution/resolved_at/post_verdict 갱신	Action	양쪽

10.6 구현: **`DiagnosticCaseStore`**

기존 `ContextVectorStore`와 동일한 패턴:

구성 요소	설명
백엔드	LanceDB (선호) / numpy fallback — `ContextVectorStore`와 동일
임베딩	`finding + likely_cause + suggested_action` 연결 텍스트를 임베딩. 온프렘: sentence-transformers 로컬 모델 (all-MiniLM-L6-v2, 384d) AWS: Bedrock Titan Embeddings V2
메타데이터 컬럼	`case_id`, `timestamp`, `agent`, `pipeline_part`, `check_item`, `verdict`, `severity`, `metrics` (JSON), `resolution`, `resolved_at`
인덱스	LanceDB IVF-PQ 인덱스 (케이스 1만건 이상 시 활성화)
보존 정책	전체 보존 (삭제 없음) — 감사 추적 요구사항. 통계 분석 시 기간 필터로 최근 N개월만 조회 가능

추천사유 ContextVectorStore와의 관계
추천사유의 `ContextVectorStore`는 고객 피처 벡터를 저장하여 사유 생성의 grounding에 사용한다 (고객 → 고객 유사성).
진단 `DiagnosticCaseStore`는 진단 리포트 텍스트를 저장하여 운영 지식으로 활용한다 (케이스 → 케이스 유사성).
같은 LanceDB 인프라를 공유하되, 별도 테이블로 분리한다.

11 구현 우선순위

Phase 0 기반	도구 인프라 + 변경 감지 — 모든 Phase의 전제 조건 `ToolRegistry` (JSON Schema 도구 정의 + 온프렘 직접 호출) `ChangeDetector` (git hook + pipeline state 이벤트 + manifest diff)
Phase 1 최우선	추천사유 품질 기반 (Audit AV3) — 비즈니스 임팩트 최고 `StratifiedReasonSampler` → `GroundingValidator` → Tier 1 집계 대시보드
Phase 2	편향 심화 분석 (Audit AV1) `IntersectionalFairnessAnalyzer` → `BiasStageAttributor`
Phase 3	운영 에이전트 + 합의 메커니즘 `OpsCollector` + `OpsDiagnoser` + `OpsReporter` `ConsensusArbiter` (독립 투표 + 2-Round 하이브리드 + 마이너리티 리포트)
Phase 4	케이스 스토어 + 지식 축적 `DiagnosticCaseStore` (LanceDB 임베딩 + 유사 검색 + 통계 분석 + 대응 효과 추적)
Phase 5	통합 + AWS Bedrock 확장 `AgentEventBridge` + `GovernanceReportGenerator` 확장 `SendNotification` + Bedrock Tool Use 연동

Listing 5: 구현 우선순위. Phase 0(기반)부터 Phase 5(통합)까지 순차 진행.

11.1 상세 구현 계획

11.1.1 태스크 목록 및 의존관계

ID	태스크	LOC	의존	리스크
	Pre-req: 추천사유 4개 갭 수정			
P-1	GAP 4: `reverse_mapper.py` 영문→한국어 fallback	30	없음	LOW
P-2	GAP 3: `interpretation_registry.py` + YAML prefix 확장	55	없음	LOW
P-3	GAP 2: InterpretationRegistry에 ReverseMapper fallback 통합	30	P-1	MED
P-4	GAP 1: `generate_l1()`에 InterpretationRegistry 3-tuple 연결	23	P-2,3	LOW
	Phase 0: 기반 인프라			
0-1	`ToolRegistry` — 38개 도구 정의 + 래퍼 + Bedrock 내보내기	450	없음	LOW
0-2	`ChangeDetector` + `_PipelineState` 콜백 + git hook	250	없음	LOW
0-3	`BaseAgent` + `agent.yaml` + `checklist.yaml` (48항목)	250	0-1	LOW
	Phase 1: 추천사유 품질 (Audit AV3)			
1-1	`StratifiedReasonSampler` — 27개 스트라텀, 과표집	180	P-4, 0-3	LOW
1-2	`GroundingValidator` — 사유↔IG top-K 정합성 + 품질 점수	150	1-1	MED
1-3	`Tier1Aggregator` — SelfChecker 결과 추이 집계	120	0-3	LOW
	Phase 2: 편향 심화 (Audit AV1)			
2-1	`IntersectionalFairnessAnalyzer` — 교차 보호속성 DI	200	0-3	MED
2-2	`BiasStageAttributor` — 단계별 DI 측정 + 증폭 식별	180	0-3	MED
	Phase 3: 운영 에이전트 + 합의			
3-1	`OpsCollector` — 7개 체크포인트 측정값 수집	250	0-1, 0-3	LOW
3-2	`OpsDiagnoser` — 연쇄 영향 룰 테이블	200	3-1	LOW

ID	태스크	LOC	의존	리스크
3-3	`OpsReporter` — 템플릿 기반 리포트 생성	150	3-2	LOW
3-4	`ConsensusArbiter` — AWS 독립투표 + 온프렘 2-Round	350	3-5	HIGH
3-5	LLM Provider 확장 — Solar, LocalLLM(Qwen/Exaone) 추가	150	없음	MED
Phase 4: 진단 케이스 스토어				
4-1	`DiagnosticCaseStore` — LanceDB + 4개 도구	300	0-1	LOW
Phase 5: 통합 + AWS 확장				
5-1	`AgentEventBridge` — 상호 트리거	120	Phase 3	LOW
5-2	`GovernanceReportGenerator` 확장 — 9개 섹션 데이터 주입	80	Phase 1-3	LOW
5-3	`SendNotification` — Slack/Email/SNS	120	0-1	LOW
5-4	`BedrockDialogSession` — Tool Use 대화 인터페이스	200	0-1, 3-5	MED
합계: 3,700 LOC / 신규 20개 파일 + 수정 5개 파일				

11.1.2 디렉토리 구조

```
core/agent/                          # 신규 패키지
    __init__.py
    base.py                          # BaseAgent ABC
    tool_registry.py                 # ToolRegistry + 도구 래퍼
    change_detector.py               # Push/Pull 변경 감지
    consensus.py                     # ConsensusArbiter
    case_store.py                    # DiagnosticCaseStore (LanceDB)
    event_bridge.py                  # AgentEventBridge
    notification.py                  # SendNotification
    bedrock_dialog.py                # BedrockDialogSession
    ops/
        __init__.py
        collector.py                 # OpsCollector (CP1-CP7)
        diagnoser.py                 # OpsDiagnoser (연쇄 분석)
        reporter.py                  # OpsReporter
    audit/
        __init__.py
        reason_sampler.py            # StratifiedReasonSampler
        grounding_validator.py       # GroundingValidator
        tier1_aggregator.py          # Tier1 SelfChecker 집계
        intersectional_fairness.py   # IntersectionalFairnessAnalyzer
        bias_stage_attributor.py     # BiasStageAttributor
        diagnoser.py                 # AuditDiagnoser
        reporter.py                  # AuditReporter

configs/financial/                   # 신규 에이전트 설정
    agent.yaml                       # 에이전트 설정
    agent_tools.yaml                 # 38개 도구 JSON Schema
    checklist.yaml                   # 48개 체크리스트 항목

scripts/hooks/
    post_commit.py                   # git hook → ChangeDetector
```

11.1.3 의존관계 그래프

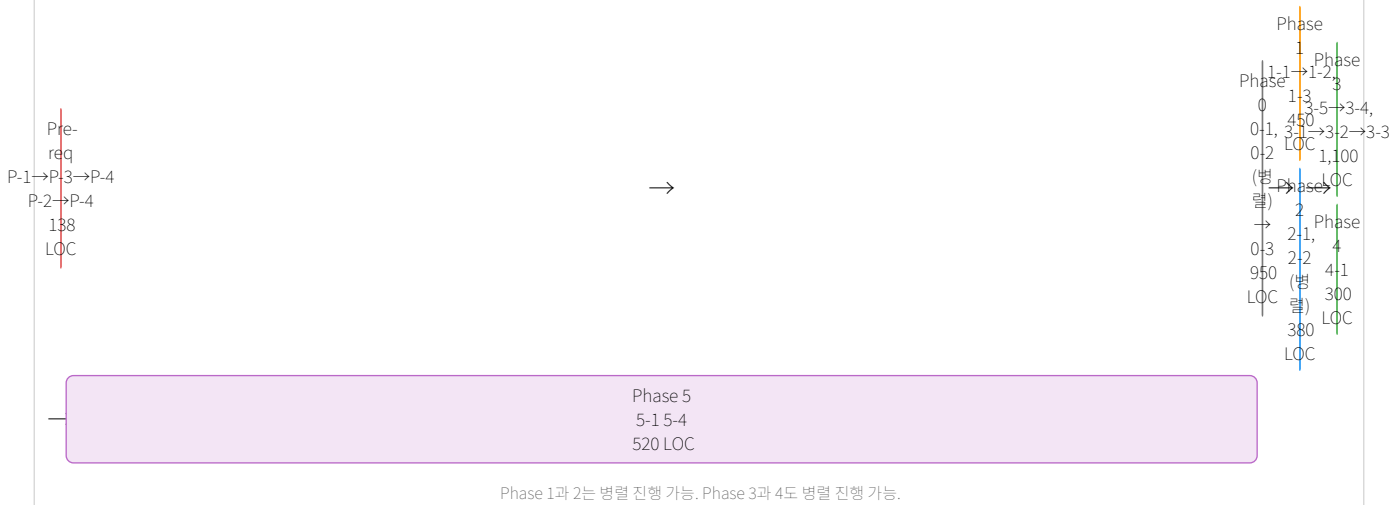


Figure 17: 의존관계 그래프. 병렬 가능한 Phase끼리는 동시 진행.

11.1.4 검증 프로토콜 (Phase별)

Phase	테스트 유형	검증 내용
Pre-req	단위	각 GAP 독립 테스트. 한국어 출력 확인. 3-tuple end-to-end 흐름.
Phase 0	단위 + 통합	ToolRegistry: mock 컴포넌트로 전체 도구 호출. ChangeDetector: 이벤트 포맷. `_PipelineState` 콜백: fire-and-forget 확인.
Phase 1	단위	StratifiedReasonSampler: 합성 데이터로 27개 스트라텀. GroundingValidator: 알려진 IG 피처 + 사유 텍스트 매칭.
Phase 2	단위	IntersectionalFairness: 알려진 DI 위반 케이스. BiasStageAttributor: 합성 파이프라인 단계별 결과.
Phase 3	단위 + Mock LLM	OpsCollector: mock 도구 호출. ConsensusArbiter: DummyProvider로 마이너리티 보존 검증. 구조화 출력 파싱 fallback.
Phase 4	단위 + 통합	DiagnosticCaseStore: `ContextVectorStore` 테스트 패턴 재사용. 검색 정확도, 저장/로드, 통계 집계.
Phase 5	End-to-end	Ops 실행 → Audit 트리거 → 케이스 저장 → 거버넌스 리포트 생성. Bedrock 연동은 실 API 테스트 (비용 발생).

12 PaperClip 선택적 차용

PaperClip (dotta, 2026.3, 3주 만에 GitHub 30K stars)은 에이전트를 "직원"으로 조직화하는 오픈소스 프레임워크이다. "zero-human company" 철학은 우리의 "AI가 분석하고 사람이 판단한다" 원칙과 충돌하므로 전면 도입은 부적합하지만, 3가지 메커니즘을 선택적으로 차용한다.

12.1 차용 1: Heartbeat 패턴 — 에이전트 주기 실행

PaperClip의 핵심 메커니즘: 매 30분(설정 가능)마다 Gateway 데몬이 에이전트를 깨우고, 에이전트는 `HEARTBEAT.md` 체크리스트를 읽어 조치가 필요한 항목만 처리한다. 조치가 불필요하면 `HEARTBEAT_OK`를 반환하고 다시 잠든다.

우리 시스템에 적용: OpsAgent의 CP5(5분 주기 서빙 헬스)와 CP6(1시간 집계)가 정확히 이 패턴이다. 현재 구현:

체크포인트	현재 방식	Heartbeat 적용 후
CP5 서빙 헬스	외부 스케줄러(EventBridge/cron)가 호출	에이전트 자체 heartbeat (5분). 정상이면 `HEARTBEAT_OK` → 무동작. 이상 시에만 체크리스트 실행 → 리포트 생성.
CP6 추천 응답	1시간 집계 배치	heartbeat 간 누적 메트릭 → 1시간마다 집계 판정. 변동 없으면 skip.
AV1 AV5 감사	일/주 단위 외부 트리거	감사 에이전트 heartbeat (일 1회). 전일 변경 없으면 `HEARTBEAT_OK` → skip.

핵심 차이: PaperClip은 에이전트가 자율적으로 무엇을 할지 결정하지만, 우리는 heartbeat가 고정된 체크리스트를 실행한다 — 자율성이 아니라 효율성을 위한 차용.

12.2 차용 2: 에이전트별 예산 캡 — "선불 직불카드" 모델

PaperClip의 예산 관리: 에이전트마다 월간 토큰 예산을 부여. 80% 도달 시 소프트 경고, 100% 도달 시 에이전트 자동 정지. 이상 소비 패턴 감지 시 예산 소진 전 회로 차단.

우리 시스템에 적용: Bedrock 비용 제어에 직접 적용 가능.

```yaml
# agent.yaml에 추가
budget:
  ops:
    monthly_token_limit: 500000    # ~$5/월
    soft_warning_pct: 0.80         # 80%에서 경고
    hard_stop_pct: 1.00            # 100%에서 정지
  audit:
    monthly_token_limit: 800000    # ~$8/월 (사유 품질 검증 포함)
    soft_warning_pct: 0.80
    hard_stop_pct: 1.00
  consensus:
    per_session_limit: 10000       # 합의 1회당 상한
    daily_limit: 50000             # 일 합의 상한
```

구현:

- `ToolRegistry.call()` 호출 시 `BudgetTracker`가 토큰 사용량 누적
- 80% 도달 → `send_notification("WARNING", "OpsAgent 예산 80% 도달")`
- 100% 도달 → 에이전트 정지, 룰 엔진만 동작 (LLM 호출 차단)
- 관리자가 수동 리셋할 때까지 LLM 기능 일시 중단

> 예산 초과 시 graceful degradation
> 예산 한도 도달 시 에이전트가 완전히 멈추는 것이 아니라,
> LLM 호출만 차단되고 룰 엔진 기반 판정은 계속 동작한다.
> 이것은 우리 아키텍처의 "온프렘 baseline이 LLM 없이 완결" 설계와 정확히 일치한다 —
> 예산 초과 = 임시 온프렘 모드.

12.3 차용 3: 전체 도구 호출 추적 (Full Trace)

PaperClip의 감사: 모든 instruction, response, tool call, decision이 불변 감사 로그에 기록된다. "어둠 속에서 일어나는 일은 없다."

우리 시스템에 이미 있는 것: `AuditLogger` (HMAC 해시체인). 추가할 것: 에이전트의 모든 도구 호출을 자동 추적.

현재 `ToolRegistry.call()`은 도구를 실행만 한다. PaperClip 차용으로 호출 전후 자동 로깅을 추가:

```python
# ToolRegistry.call() 확장
def call(self, name, params=None):
    # Before: log intent
    trace = {"tool": name, "params": params, "timestamp": now()}

    result = tool.func(**(params or {}))

    # After: log result
    trace["result_summary"] = summarize(result)
    trace["token_cost"] = estimate_tokens(params, result)
    self._trace_log.append(trace)
    self._budget_tracker.add(trace["token_cost"])

    return result
```

이 추적 데이터는 DiagnosticCaseStore와 별도로 에이전트 활동 로그로 저장되어, "이 진단 결과가 어떤 도구 호출을 거쳐 생성되었는가"를 완전히 재현 가능하게 한다.

12.4 차용하지 않는 것과 그 이유

PaperClip 메커니즘	미차용 이유	우리의 대안
에이전트가 에이전트를 고용	감사 관점에서 자율 에이전트 생성은 위험. "누가 이 에이전트를 만들었는가"에 답할 수 없다.	고정된 2개 에이전트(Ops/Audit) + YAML 체크리스트
SOUL.md 페르소나	에이전트에 "성격"을 부여하면 일관성이 깨질 수 있다.	시스템 프롬프트가 역할(보수적/통계적/비즈니스)을 정의 — 체크리스트 판정에만 영향
자율 의사결정	EU AI Act Art.14 인간 감독 위반. 에이전트의 자율 조치는 금융 규제에서 허용 불가.	에이전트는 권고만, 최종 결정은 사람. Action 도구도 승인 후 실행.
Node.js 서버	Python 생태계와 불일치. PyTorch/DuckDB/LanceDB 모두 Python.	Python 네이티브 구현 (`core/agent/`)
PARA 메모리 시스템	파일 기반 메모리는 구조화 검색이 어렵다.	LanceDB DiagnosticCaseStore — 벡터 검색 + 구조화 쿼리 동시 지원

13 메모리 프레임워크 선택적 차용

2026년 초 여러 에이전트 메모리 프레임워크(Mem0, Zep/Graphiti, Letta/MemGPT, SuperLocalMemory, LangMem 등)가 발표되었다. 우리 시스템은 이미 상당 부분의 메모리 인프라를 보유하고 있으므로, 프레임워크를 통째로 도입하지 않고 핵심 알고리즘/패턴만 차용한다.

13.1 이미 해결된 것과 실제 갭

기능	현재 상태	실제 갭
케이스 축적	`DiagnosticCaseStore` (LanceDB) 이미 구현	시간 decay 없음 — 3년 전 케이스와 어제 케이스가 동일 가중치
피처 해석	`InterpretationRegistry` 5-level cascade 이미 구현	고객 서술적 프로파일("적금 선호, 리스크 회피") 없음
감사 추적	HMAC 해시체인 + S3 Object Lock 7년 이미 구현	시점 T 스냅샷 복원 비효율 — 여러 컴포넌트 조인 필요

기능	현재 상태	실제 갭
담당자 대화	`BedrockDialogSession` 세션 기반	세션 종료 시 대화 이력 소실 — "지난번 논의한 이슈" 참조 불가

13.2 차용 결정

순위	프레임워크	차용 내용	우선순위
1	Zep/Graphiti	시간적 지식 그래프 패턴: `(entity, attribute, value, valid_from, valid_to)` 스키마로 "시점 T 스냅샷 복원" 쿼리를 단일 필터로 처리	HIGH
2	SuperLocalMemory	수학적 decay 메커니즘: `exp(-age/τ)` 가중치를 유사 검색에 적용. 원본은 보존(규제 요건), 검색 가중치만 조정.	HIGH
3	Mem0	팩트 압축 레이어: 배치 시점에 고객 피처 → 서술적 팩트 추출. 룰 기반으로 구현하여 LLM 호출 없이 L2a 프롬프트 강화.	MEDIUM
4	Letta (MemGPT)	Recall memory 패턴: 담당자 대화 이력을 DynamoDB에 저장하여 세션 간 맥락 유지. BedrockDialogSession에 통합.	MEDIUM
–	LangMem	차용하지 않음 — 프롬프트 자기개선은 감사 관점에서 위험. "누가 이 프롬프트를 승인했는가"에 답할 수 없음. 우리 원칙 "AI가 분석하고 사람이 판단한다"에 어긋남.	SKIP
–	Succession/ALE	차용하지 않음 — Claude Code 세션 수명이 짧아 현재 불필요.	SKIP

13.3 차용 1: 시간적 지식 그래프 (Zep/Graphiti 패턴)

감사 질의 "2026-03-15 시점에 고객 A에게 펀드 X를 추천한 근거는?"에 답하려면 그 시점의 모델 버전, 피처 스냅샷, 체크리스트 상태, 에이전트 판정을 동시에 복원해야 한다. 현재는 각 컴포넌트에 분산되어 있어 조인 비용이 크다.

13.3.1 LanceDB 스키마 (신규 **`TemporalFactStore`**)

```python
class TemporalFactStore:
    """시간적 유효 범위가 있는 팩트 저장소.

    DiagnosticCaseStore와 같은 LanceDB 백엔드 재사용.
    """

    schema = {
        "fact_id": str,
        "entity_type": str,     # "customer", "model", "recommendation", "checklist"
        "entity_id": str,
        "attribute": str,        # "segment", "version", "verdict"
        "value": str,            # JSON-serialized
        "valid_from": datetime,
        "valid_to": datetime,    # None = 현재 유효
        "source": str,           # "pipeline", "agent", "operator"
        "vector": List[float],   # 자연어 설명 임베딩 (선택)
    }
```

13.3.2 대표 쿼리

```python
# "시점 T의 고객 A 모든 팩트"
store.snapshot_at(entity_id="cust_A", at_time="2026-03-15T00:00:00Z")
# → SELECT * FROM facts
#    WHERE entity_id = 'cust_A'
#      AND valid_from <= '2026-03-15T00:00:00Z'
#      AND (valid_to IS NULL OR valid_to > '2026-03-15T00:00:00Z')
```

```
# "시점 T의 모델 상태"
store.snapshot_at(entity_type="model", at_time="2026-03-15")
```

대부분의 감사 쿼리가 단일 엔티티의 시점 복원이라 LanceDB 네이티브 필터로 해결 가능. JOIN 불필요.

13.3.3 구현 위치

`core/agent/case_store.py`에 `TemporalFactStore` 클래스 추가 (기존 `DiagnosticCaseStore`와 같은 LanceDB 인스턴스 공유).

13.4 차용 2: 수학적 Decay (SuperLocalMemory 패턴)

현재 `DiagnosticCaseStore.search_similar()`는 모든 케이스를 동일 가중치로 검색한다. 하지만 실무적으로 최근 케이스가 더 관련성 높다 — 3년 전 drift 해결 방식이 지금도 유효한지 불확실하다.

13.4.1 Decay 함수

$$\text{weight}(c) = \text{cosine_similarity}(c) \times \exp\left(-\frac{\text{age_days}(c)}{\tau}\right)$$

여기서 τ는 반감기 (예: 90일 → 3개월 전 케이스는 가중치 절반).

13.4.2 중요: 삭제가 아님

- 원본 케이스는 보존 (HMAC 감사 로그 7년 보존 요건)
- 검색 가중치만 조정
- 규제기관이 "3년 전 케이스를 왜 삭제했는가?"라고 물을 여지 없음

13.4.3 구현 위치

```
# core/agent/case_store.py 수정
def search_similar(
    self,
    query_vector: np.ndarray,
    k: int = 5,
    pipeline_part: Optional[str] = None,
    decay_half_life_days: Optional[float] = 90.0,  # 신규
) -> List[Tuple[Dict, float]]:
    ...
    # 기존: similarity = cosine(query, case.vector)
    # 신규:
    age_days = (now - case.timestamp).days
    decay = math.exp(-age_days / decay_half_life_days) if decay_half_life_days else 1.0
    adjusted_score = similarity * decay
    ...
```

30 LOC 추가로 구현 가능.

13.5 차용 3: 고객 팩트 압축 (Mem0 패턴)

현재 L2a 사유 생성 시 IG top-K 피처만 프롬프트에 전달된다. 고객의 서술적 프로파일("적금 선호", "최근 펀드 관심 증가")이 없어 Solar Pro나 Qwen 14B가 맥락 부족 상태로 사유를 생성한다.

13.5.1 룰 기반 FactExtractor (LLM 없음)

```
# core/recommendation/reason/fact_extractor.py (신규)
class FactExtractor:
    """고객 피처로부터 서술적 팩트 추출 (룰 기반).

    Mem0와 달리 LLM 호출 없이 결정론적으로 동작.
    """
```

```
    def extract(self, customer_features: Dict) -> List[str]:
        facts = []
        # 상품 보유 패턴
        if customer_features.get("deposit_balance_ratio", 0) > 0.6:
            facts.append("예적금 중심 포트폴리오")
        # 최근 관심사
        if customer_features.get("fund_view_count_3m", 0) > 5:
            facts.append("최근 3개월 펀드 관심 증가")
        # 리스크 성향
        risk_score = customer_features.get("risk_tolerance_score", 0.5)
        if risk_score < 0.3:
            facts.append("리스크 회피 성향")
        # ... config 기반 규칙
        return facts
```

13.5.2 통합 방식

Phase 0 배치에서 미리 추출하여 `customer_facts` 컬럼으로 LanceDB에 저장. 서빙 타임에는 조회만 → LLM 호출 없이 프롬프트 강화.

`InterpretationRegistry`와 별도로 작동하는 고객 레벨 메모리 레이어이다.

13.6 차용 4: Dialog Recall Memory (Letta 패턴)

`BedrockDialogSession`은 대화 이력을 메모리에만 보관하여 세션 종료 시 소실된다. 담당자가 "지난주 논의한 그 drift 이슈 기억해?"라고 물어도 에이전트가 답할 수 없다.

13.6.1 구현

DynamoDB에 대화 이력 저장:

```
# core/agent/bedrock_dialog.py 확장
class BedrockDialogSession:
    def __init__(self, ..., recall_memory: Optional["DialogRecallMemory"] = None):
        self._recall = recall_memory

    def chat(self, user_message: str) -> str:
        # 이전 세션의 관련 대화 조회
        if self._recall:
            past_context = self._recall.search_related(user_message, limit=5)
            # 시스템 프롬프트에 첨부
            ...

        response = super().chat(user_message)

        # 저장
        if self._recall:
            self._recall.save_turn(user_message, response)

        return response
```

`DialogRecallMemory`는 DynamoDB 테이블 `agent_dialog_recall`에 `(operator_id, session_id, turn_id, user_msg, agent_response, timestamp, embedding)` 저장. 검색은 임베딩 기반 유사도.

13.7 구현 우선순위 (메모리 프레임워크 차용)

ID	태스크	LOC	의존	Phase
M-1	DiagnosticCaseStore `search_similar()`에 decay 추가	30	없음	즉시
M-2	`TemporalFactStore` (시간 그래프)	120	LanceDB	Phase 4 확장

ID	태스크	LOC	의존	Phase
M-3	`FactExtractor` (룰 기반 고객 팩트)	150	feature_groups.yaml	Phase 1 확장
M-4	`DialogRecallMemory` + BedrockDialog 통합	80	DynamoDB	Phase 5 확장

총 380 LOC. 기존 아키텍처를 크게 건드리지 않고 증분 추가.

> LanceDB 단일 백엔드 원칙
> M-1, M-2, M-3 모두 기존 `DiagnosticCaseStore`/`ContextVectorStore`와
> 같은 LanceDB 인스턴스를 공유한다.
> DuckDB 등 새 의존성을 추가하지 않으며, 벡터 검색 + 메타데이터 필터를
> 단일 스택에서 처리한다. M-4만 DynamoDB를 사용 (기존 reason_cache와 동일 스택).

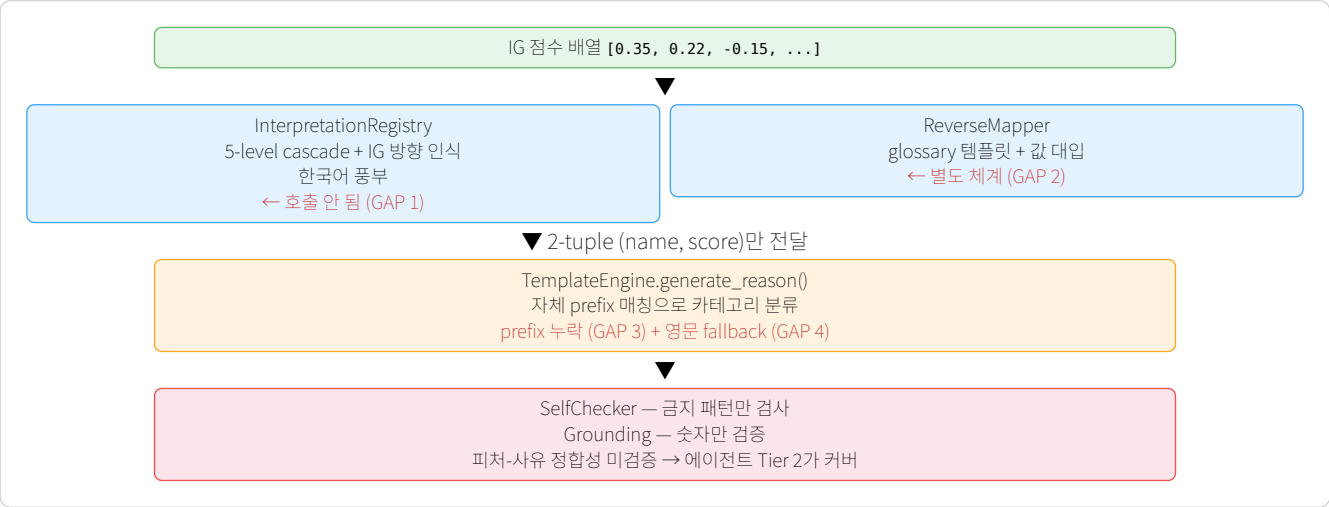
14 미결 설계 과제

과제	상세
Tier 3 전문가 리뷰 UI	리뷰 인터페이스를 어디에 만들 것인가 (Retool, 사내 도구, Streamlit 등)
샘플링 비율 자동 조정	Tier 2 품질 저하 시 샘플 비율을 자동으로 5%→10%로 올릴 것인가
교차속성 조합 폭발	보호속성 5개의 2-way 조합 = 10개, 3-way = 10개 — 어디까지 분석할 것인가
진단 정확도 검증	“likely_cause” 추정이 실제 원인과 일치했는지 사후 검증하는 메커니즘
Tier 2 GPU 비용	perturbation 기반 XAI 평가의 GPU 비용 vs 감사 가치 trade-off
에이전트 자체 모니터링	에이전트가 정상 작동하지 않을 때의 감시 (watchdog)
FeatureStore latency 계측	`health_check()`에 per-request latency 없음. DynamoDB는 CloudWatch `SuccessfulRequestLatency`로 대체 가능하나, Memory 백엔드는 계측 추가 필요.
Git hook 인프라 구축	현재 `.git/hooks/`에 샘플만 존재. `post-commit` hook으로 ChangeDetector에 이벤트 전달하는 구조를 신규 구축해야 함. pre-commit framework 또는 raw hooks 중 선택 필요.
`_PipelineState` 이벤트 발행	`mark_complete()`에 콜백/이벤트 메커니즘 없음. `__init__`에 `callbacks: List[Callable]` 추가하여 완료 시 에이전트에 알림. 기존 코드 영향 없음 (EASY).
`verify_chain()` 증분 검증	현재 GENESIS부터 전체 체인만 검증 가능. 에이전트가 매번 전체 파일을 읽지 않으려면 `start_hash` 파라미터 추가 필요.

15 추천사유 생성 파이프라인 연동 — 갭 분석 및 구현 계획

에이전트 설계와 추천사유 생성 코드의 연동을 점검한 결과, 각 컴포넌트는 프로덕션 수준이지만 컴포넌트 간 연결에 4개 갭이 발견되었다. 모두 코드 수정으로 해결 가능하며, 에이전트 연동의 전제 조건이다.

15.1 현재 흐름과 갭



Listing 6: 현재 흐름과 4개 갭. 빨간 텍스트가 수정 대상.

15.2 4개 갭 상세

GAP	심각도	내용	수정 파일
1	HIGH	`generate_l1()`이 `InterpretationRegistry`를 호출하지 않음. 풍부한 한국어 해석(IG 방향 + 태스크 맥락)이 사용되지 않고 2-tuple `(name, score)`만 template engine에 전달.	`async_orchestrator.py`
2	HIGH	`ReverseMapper`와 `InterpretationRegistry`가 병렬 체계로 독립 존재. 호출자가 어느 쪽을 쓸지 알아서 선택해야 함. 통합 필요.	`interpretation_registry.py` `pipeline.py`
3	MEDIUM	`_DEFAULT_PREFIX_TO_GROUP`에 `spend_`, `amount_`, `age_`, `tenure_`, `product_`, `card_` 등 12개+ prefix 누락. 그룹 매핑 실패 → raw 텍스트 출력.	`interpretation_registry.py` `feature_groups.yaml`
4	MEDIUM	`ReverseMapper` 매핑 실패 시 영문 fallback `"Unknown/feature_42 is above average."` 한국어 사유 중간에 영문 혼입.	`reverse_mapper.py`

15.3 구현 순서 (의존관계 기반)

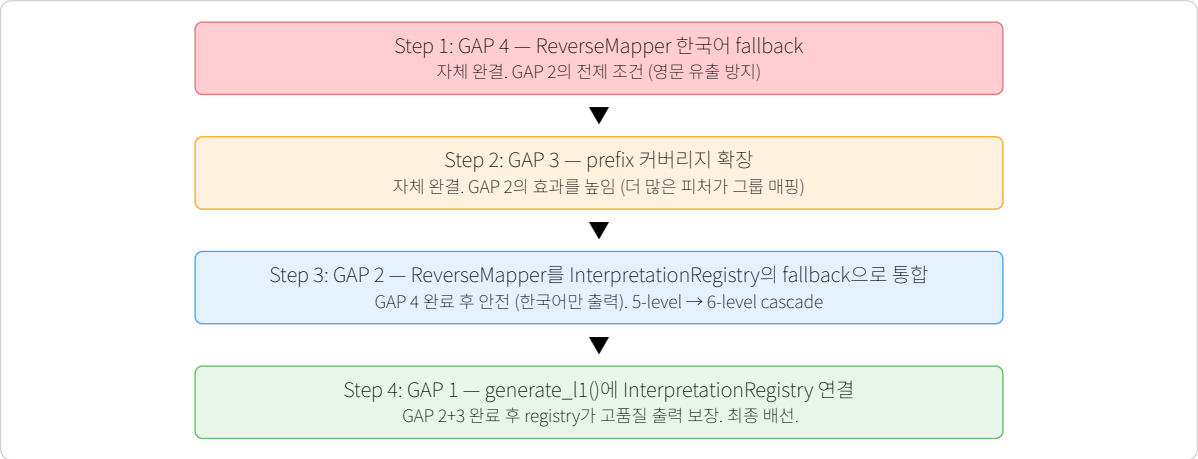


Figure 18: 구현 순서. 아래로 갈수록 이전 Step에 의존.

15.4 수정 상세

15.4.1 Step 1: GAP 4 — ReverseMapper 한국어 fallback

파일: `core/recommendation/reason/reverse_mapper.py`

- 기본 `interpretation_templates` (영문) → 한국어 교체:
  - `"very_low"` → `"해당 지표({feature_label})가 매우 낮은 수준입니다."`
  - `"low"` → `"해당 지표({feature_label})가 평균보다 낮은 수준입니다."`
  - `"medium"` → `"해당 지표({feature_label})가 보통 수준입니다."`
  - `"high"` → `"해당 지표({feature_label})가 평균보다 높은 수준입니다."`
  - `"very_high"` → `"해당 지표({feature_label})가 매우 높은 수준입니다."`
- `_render_interpretation()` 영문 fallback → 한국어 교체
- `"Unknown"` 그룹 라벨 → `"미분류 그룹"` 교체
- config에서 `interpretation_templates`를 오버라이드할 수 있도록 유지 (config-driven)

15.4.2 Step 2: GAP 3 — prefix 커버리지 확장

파일: `core/recommendation/reason/interpretation_registry.py`, `configs/santander/feature_groups.yaml`

- `feature_groups.yaml`에 `prefix_to_group` 섹션 추가 (config-driven):

```yaml
prefix_to_group:
  spend_: base_txn_stats
  amount_: base_txn_stats
  age_: demographics
  tenure_: demographics
  product_: product_holdings
  card_: product_holdings
  ...
```

- `from_configs()`에서 YAML의 `prefix_to_group`을 읽어 `_DEFAULT_PREFIX_TO_GROUP`에 병합
- prefix 매칭 시 longest-first 정렬 (ambiguity 방지)
- `_auto_generate_level1()`의 `group_semantics`에 신규 그룹 한국어 설명 추가

15.4.3 Step 3: GAP 2 — ReverseMapper를 InterpretationRegistry fallback으로 통합

파일: `core/recommendation/reason/interpretation_registry.py`, `core/recommendation/pipeline.py`

- `InterpretationRegistry.__init__()`에 `reverse_mapper` 파라미터 추가 (Optional, 기본 None)
- `interpret()` cascade에 Level RM 추가 (L1과 glossary fallback 사이):

```
IG → L3 → L2 → L1 → RM(신규) → glossary fallback
```

- `ReverseMapper.interpret_financial()`의 한국어 결과를 반환 (GAP 4에서 한국어 보장)
- `RecommendationPipeline`에서 InterpretationRegistry 생성 시 ReverseMapper 인스턴스 주입

15.4.4 Step 4: GAP 1 — generate_l1()에 InterpretationRegistry 연결

파일: `core/recommendation/reason/async_orchestrator.py`

- `__init__()`에 `interpretation_registry` 파라미터 추가 (Optional, 기본 None)
- `generate_l1()`에서 registry가 있으면:

```
interpreted = self._interpretation_registry.interpret_batch(
    features=features, task=task_type or ""
)
enriched = [(e["name"], e["value"], e["text"]) for e in interpreted]
```

- enriched 3-tuple을 `TemplateEngine.generate_reason()`에 전달
- registry가 None이면 기존 2-tuple 동작 유지 (하위 호환)

15.5 인터페이스 계약 검증 표

생산자	출력	소비자	기대 입력
`InterpretationRegistry.interpret_batch()`	`List[Dict]` keys: name, value, text	`AsyncReasonOrchestrator.generate_l1()`	3-tuple `(name, value, text)`로 변환
3-tuple	`(str, float, str)`	`TemplateEngine._ig_based_reasons()`	`len(entry) >= 3` 체크 (이미 구현)
`ReverseMapper.interpret_financial()`	`str` (한국어)	`InterpretationRegistry.interpret()` Level RM	비어있지 않은 한국어 문자열
`feature_groups.yaml prefix_to_group`	`Dict[str, str]`	`InterpretationRegistry.from_configs()`	defaults에 병합

15.6 리스크 평가

GAP	리스크	원인	완화
1	LOW	순수 추가. `interpretation_registry=None`이면 기존 동작	registry 유무 양쪽 테스트
2	MEDIUM	영문 탐지 guard 조건이 fragile	GAP 4를 먼저 구현하면 ReverseMapper가 항상 한국어 → guard 불필요
3	LOW	순수 추가. prefix 확장만	longest-first 정렬로 모호성 방지
4	MEDIUM	기본 언어 변경. 영문 패턴 의존 코드가 있을 수 있음	구현 전 영문 패턴 grep 스캔

16 Bedrock 인프라 공유 및 태스크별 모델 선택

추천사유 생성(L2a)과 운영/감사 에이전트가 동일한 Bedrock 인프라를 공유한다. 태스크 특성에 따라 최적 모델이 다르므로, 용도별 모델을 분리 배정한다.

16.1 태스크별 모델 선택 전략

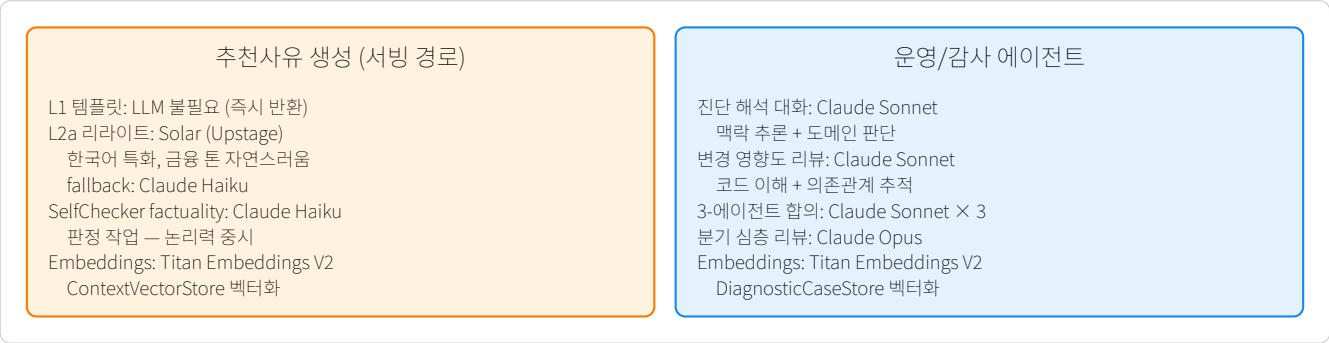


Figure 19: 태스크별 Bedrock 모델 배정. 추천사유는 한국어 특화(Solar), 에이전트는 추론력(Sonnet).

16.1.1 모델 선택 근거

태스크	모델	선택 이유	입출력	건당 비용
L2a 사유 생성	Solar (Upstage)	한국어 특화. 금융 존댓말 톤(“ 권해드립니다”) 자연스러움. Upstage가 한국 회사로 한국어 학습 데이터 풍부.	입 600 / 출 80	$0.002
L2b self-critique	Solar (Upstage)	생성 모델 ≤ 크리틱 모델 원칙. Solar가 쓴 한국어의 자연스러움, 금소법 위반, 피처 정합성을 한국어에 강한 모델이 판단해야 함. 같은 모델이지만 프롬프트가 다르므로 관점이 다름.	입 800 / 출 100	$0.003
SelfChecker factuality	Claude Haiku	factual_score(0 1) 수치 판정. 한국어 뉘앙스보다 논리적 판단이 핵심.	입 800 / 출 50	$0.001
에이전트 대화	Claude Sonnet	수치 맥락 추론 + 도메인 판단 + 코드 이해. “이 DI가 필터 문제인지 모수 문제인지” 수준의 추론 필요.	입 2K / 출 500	$0.02
3-에이전트 합의	Claude Sonnet × 3	독립 관점 투표. 각 에이전트가 충분한 추론력 필요. Haiku는 판정 근거(reasoning)의 풍부함이 부족.	입 1K / 출 600	$0.03
분기 심층 리뷰	Claude Opus	다중 규제 프레임워크 교차 분석. 분기 1회라 비용 무관.	입 5K / 출 2K	$0.30
벡터 임베딩	Titan Embeddings V2	ContextVectorStore + Diagnostic-CaseStore 공유.	입 200	$0.0001

16.1.2 한국어 추천사유에 Solar를 선택하는 이유

추천사유 L2a의 핵심 요구사항은 “고객이 납득할 만한 자연스러운 한국어 1 2문장”이다.

기준	Solar (Upstage)	Claude Haiku	Llama 3.1
한국어 자연스러움	최상 — 한국어 특화 학습	상 — 범용 다국어	중하 — 영어 중심

기준	Solar (Upstage)	Claude Haiku	Llama 3.1
금융 존댓말 톤	최상 — 한국 금융 문서 학습	상 — 범용	하 — 부자연
“ 드립니다” 어미	자연스러움	대체로 자연	어색한 경우 잦음
비용	저	저	최저
Bedrock 가용성	Marketplace (확인 필요)	네이티브	네이티브

Solar 가용성 확인 필요
Solar는 AWS Marketplace를 통해 Bedrock에서 호출 가능하나,
`ap-northeast-2` (서울) 리전 가용성을 사전 확인해야 한다.
불가 시 Claude Haiku로 fallback — 한국어 품질은 충분하지만
금융 특화 톤에서 Solar보다 약간 뒤진다.
`llm_provider.py`의 팩토리 패턴이 이미 프로바이더 교체를 지원하므로
config 변경만으로 전환 가능.

16.2 L2a 처리 아키텍처

추천사유 L2a는 고객별 순차 생성이 아니라 비동기 배치 처리로 설계되어 있다. `AsyncReasonOrchestrator`가 SQS + DynamoDB 기반으로 이미 구현.

Figure 20: L2a 비동기 처리. 고객은 항상 L1을 즉시 받고, L2a는 백그라운드에서 처리.

16.2.1 처리량 추정 (Santander 기준)

구성	대상	소요시간	비용
전체 941K 중 L2a 대상 (5%)	47K건	–	–
Solar 워커 1대 (순차)	47K × 50ms	40분	$0.10
Solar 워커 5대 (병렬)	47K × 50ms / 5	8분	$0.10
Bedrock Batch Inference	일괄 제출	수분	$0.10

입력 600 토큰, 출력 80 토큰 (1 2문장 한국어). 워커 5대 병렬이면 8분, Batch Inference면 수분 내 완료. 비용은 47K건 전체가 $0.10 미만.

16.3 Bedrock 인프라 공유 시 고려사항

고려사항	문제	대응
쿼터 경쟁	L2a 배치(47K건)와 에이전트 합의(3× 동시)가 Bedrock API 쿼터를 경쟁	L2a는 야간/주기 배치, 에이전트는 점검 직후 실행 → 시간대 분리. 또는 별도 프로비저닝.
비용 통합 추적	추천사유 비용 + 에이전트 비용을 합산 관리 필요	CloudWatch 메트릭에 `usage_type` 태그 (reason_l2a / agent_consensus / agent_dialog) → 용도별 비용 분리 추적

고려사항	문제	대응
fallback 전략	Bedrock 장애 시 추천사유와 에이전트 모두 영향	추천사유: L1 템플릿으로 fallback (이미 구현) 에이전트: 룰 엔진만으로 작동 (이미 설계) → 양쪽 모두 LLM 없이 기본 기능 유지
모델 관리	Solar + Haiku + Sonnet + Opus + Titan = 5개 모델	`llm_provider.py` 팩토리 패턴으로 용도별 프로바이더 주입. config에서 `reason.llm_model`, `agent.llm_model` 분리 관리.

16.4 온프렘 모델 선택: Exaone 3.5

온프렘에서는 Bedrock을 쓸 수 없으므로 오픈소스 한국어 특화 모델을 사용한다. LG AI Research의 Exaone 3.5 (Apache 2.0)가 최적의 선택지이다.

모델	파라미터	VRAM	한국어	비고
Exaone 3.5 7.8B	7.8B	8GB	상	RTX 4070 (12GB)에 여유롭게 올라감. 한국어 특화 학습. 같은 크기 Llama/Qwen보다 한국어 자연.
Exaone 3.5 2.4B	2.4B	3GB	중상	초경량. 품질은 7.8B보다 낮지만 속도 빠름.
Qwen 2.5 14B Q4	14B Q4	9GB	중상	범용 다국어. 한국어는 Exaone보다 약간 부자연.
Llama 3.1 8B	8B	8GB	중하	영어 중심. 한국어 부자연스러움 빈번.

온프렘 용도별 배정:

용도	모델	이유
추천사유 L2a 생성/critique	Exaone 3.5 7.8B (8GB)	한국어 자연스러움이 핵심. 금융 존댓말 톤이 Qwen/Llama보다 자연. 오픈소스(Apache 2.0).
에이전트 2-Round 합의 (R1 독립 투표 + R2 심의)	Qwen 2.5 14B Q4 (9GB)	논리력/추론력이 핵심. 한국어 자연스러움은 불필요 — 수치 해석, 인과 추론, 논리적 근거 작성이 요구됨. 파라미터 2× → Exaone 7.8B보다 추론력 높음.
임베딩 (DiagnosticCaseStore)	sentence-transformers (all-MiniLM-L6-v2)	384d, 경량, LLM과 별도 로드 가능.

VRAM 관리: 순차 로딩
Exaone 8GB + Qwen 9GB = 17GB > RTX 4070 12GB — 동시 로딩 불가.
온프렘은 배치 실행이므로 순차 로딩으로 해결:
(1) 룰 엔진 체크리스트 실행 (GPU 불필요)
(2) Qwen 14B 로드 → 에이전트 합의 → 언로드
(3) Exaone 7.8B 로드 → 추천사유 L2a 배치 → 언로드
모델 로딩/언로딩에 30초 소요, 전체 흐름에 영향 미미.

Exaone 생태계 동향
LG AI Research는 Exaone 3.5 이후 K-Exaone (236B MoE, 23B active)과
Exaone 4.5 (멀티모달)를 연이어 출시.
K-Exaone은 글로벌 AI 벤치마크 7위로 Qwen3, GPT를 상회하는 성적.

향후 K-Exaone이 오픈소스화되면 온프렘 품질이 한 단계 더 올라갈 수 있다.
현재 온프렘에서 RTX 4070으로 돌릴 수 있는 것은 3.5 7.8B가 최선.

17 규제 충족 매핑

이 시스템의 핵심 설계 원칙은 "AI가 분석하고, 사람이 판단한다"이다. 에이전트의 역할은 의사결정이 아니라 의사결정 보조로 명확히 한정된다. 이 구조가 주요 규제 프레임워크의 요구사항을 어떻게 충족하는지 매핑한다.

17.1 EU AI Act

조항	요구사항	충족 방식	근거 구성요소
Art.9	리스크 관리 시스템	3-에이전트 합의로 할루시네이션 리스크 구조적 완화. 마이너리티 리포트로 소수 의견까지 보존.	합의 메커니즘 + 마이너리티 리포트
Art.12	기록 보관 (로깅)	모든 진단 결과가 케이스 스토어(LanceDB) + HMAC 해시 체인 감사 로그(S3 Object Lock 7년)에 이중 기록. 에이전트별 reasoning 전문 보존.	DiagnosticCaseStore + AuditLogger
Art.13	투명성 · 설명 가능성	에이전트가 verdict뿐 아니라 풍부한 reasoning(300 600 토큰)을 출력. 판단 근거가 명시적으로 기록되어 사후 검증 가능.	에이전트 출력 사양 (reasoning 필드)
Art.14	인간 감독	에이전트는 권고만 하고 자동 조치 없음. Action 도구도 명시적 승인 후에만 실행. 최종 판단은 반드시 담당자가 수행.	Query/Action 분리 + 승인 메커니즘
Art.15	정확성 · 견고성	독립 투표(Round 1)로 단일 모델 편향 방지. 룰 엔진(결정론적) + LLM(확률적)의 이중 구조로 견고성 확보.	2-Round 하이브리드 + 룰 엔진

17.2 한국 금감원 AI 가이드라인

요구사항	충족 방식	근거 구성요소
설명 가능성	풍부한 reasoning + 마이너리티 리포트까지 포함하여 "모든 관점을 검토했음"을 증명	에이전트 출력 + 마이너리티 리포트
공정성 모니터링	48개 체크리스트 중 공정성 항목(4.6, 4.7, 4.8, 4.9)을 정기 자동 점검 + 교차 보호속성 분석	체크리스트 + IntersectionalFairnessAnalyzer
감사 추적	HMAC 해시 체인 불변 감사 로그 + 케이스 스토어 이력 + 거버넌스 리포트 월/분기 자동 생성	AuditLogger + DiagnosticCaseStore + GovernanceReport
인간 개입	에이전트 산출물을 담당자가 읽고 판단하고 점검하는 구조. 합의 결과도 최종 판정이 아니라 리뷰 우선순위 분류일 뿐.	전체 아키텍처 설계 원칙

요구사항	충족 방식	근거 구성요소
모델 리스크 관리	3-에이전트 합의 + 케이스 스토어 대응 효과 추적 + 마이너리티 적중률 사후 검증	합의 메커니즘 + 케이스 스토어 피드백 루프

17.3 한국 AI 기본법

요구사항	충족 방식	근거 구성요소
고영향 AI 분류 대응	FRIA 5차원 리스크 평가 자동 실행. 감사 에이전트 AV4에서 주기적 점검.	FRIAEvaluator + 체크리스트 6.6
AI 이용자 권리	AI 결정 거부(opt-out) 이력 관리. 추천사유에 AI 공시 문구 포함 확인(체크리스트 5.10).	opt_out_audit + SelfChecker
킬스위치 / 인간 감독	킬스위치 상태 모니터링(CP5). 에이전트는 킬스위치 권고만, 활성화는 사람이 수행.	체크리스트 4.4 + Action 도구 승인

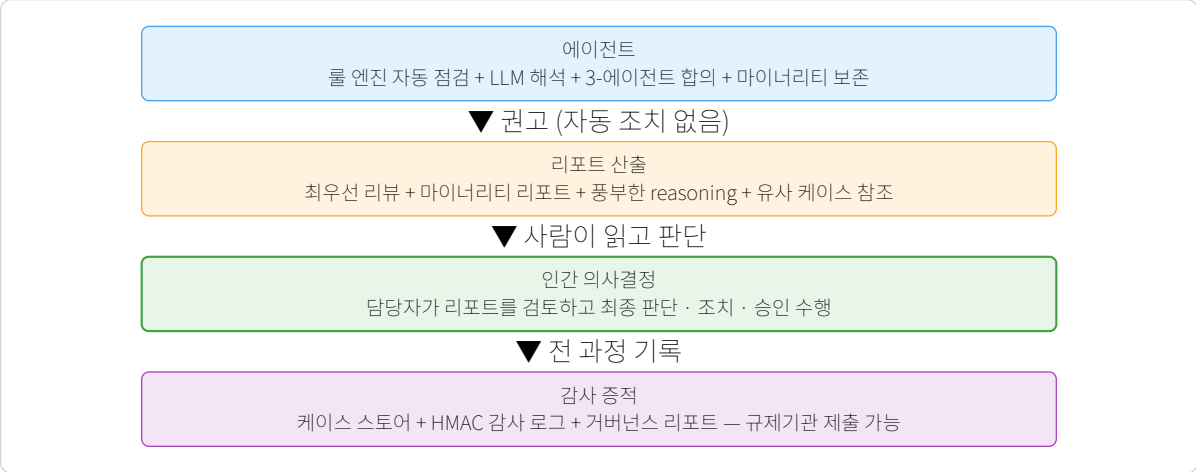


Figure 21: 규제 충족 흐름. AI 분석 → 사람 판단 → 감사 증적의 3단계가 규제 요구사항을 구조적으로 충족.

왜 이 구조가 규제적으로 안전한가
"AI가 판단하고 AI가 조치한다"가 아니라 "AI가 분석하고 사람이 판단한다"는 구조이다.
에이전트는 (1) 자동 점검, (2) 이상 징후 해석, (3) 다중 관점 합의, (4) 소수 의견 보존까지만 책임지고,
최종 의사결정과 조치는 반드시 사람이 수행한다.
이 경계가 명확하기 때문에 EU AI Act의 "인간 감독" 요건,
금감원의 "인간 개입" 요건, AI 기본법의 "킬스위치" 요건을 모두 충족한다.